



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that ___Jai_Talreja___ of D20A semester VIII, have successfully completed necessary experiments in the ITC801: Blockchain & DLT Lab under my supervision in **VES Institute of Technology** during the academic year 2025-2026.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph

Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Exp 1

Aim :- To Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash.

Theory :-

1. Cryptographic Hash Functions in Blockchain

Cryptographic hash functions are algorithms that take input data of any size and generate a fixed-length, irreversible hash. In blockchain, SHA-256 is commonly used to secure transactions and blocks. These functions are deterministic, collision-resistant, and produce a completely different output even with a small change in input. Hash functions ensure data integrity, link blocks together securely, and prevent tampering, forming the backbone of blockchain security.

2. Merkle Tree

A Merkle Tree is a binary tree structure that efficiently summarizes and verifies large amounts of data using cryptographic hashes. In a Merkle Tree, the leaf nodes represent the hashes of individual transactions, while the parent nodes are hashes of their child nodes combined. The topmost node, called the Merkle Root, uniquely represents all the transactions in the block. This structure is widely used in blockchain to secure transaction integrity.

3. Structure of Merkle Tree

- Leaf Nodes: Hash of individual transactions
- Intermediate Nodes: Hash of two child nodes combined
- Merkle Root: Final single hash at the top

If the number of nodes is odd, the last hash is duplicated.

4. Merkle Rule

The Merkle rule states that each parent node in the tree is computed as the hash of its two child nodes. If there is an odd number of nodes at any level, the last node is duplicated before hashing. This process continues recursively until a single hash, called the Merkle Root, is obtained. This rule ensures that the root hash uniquely represents all underlying transactions and any modification in a transaction can be quickly detected.

5. Working of Merkle Tree

The working of a Merkle Tree involves first hashing all individual transactions to create leaf nodes. Then, pairs of hashes are combined and hashed repeatedly to form intermediate nodes. This process continues until only the Merkle Root remains at the top of the tree. The Merkle Root serves as a single, verifiable summary of all transactions, allowing efficient verification without needing to examine every transaction individually.

6. Benefits of Merkle Tree

- Efficient data verification
- Reduced storage requirements
- Fast integrity checking
- Tamper detection
- Scalable for large datasets

7. Use of Merkle Tree in Blockchain

In blockchain, Merkle Trees are used to securely store transactions in each block and generate the Merkle Root. This allows nodes to verify transactions efficiently without downloading the entire block, which is especially useful for lightweight clients. Merkle Trees also ensure data integrity, prevent tampering, and maintain a verifiable link between all transactions within a block.

8. Use Cases of Merkle Tree

- Blockchain transaction verification
- Distributed systems
- File integrity verification
- Version control systems (Git)
- Peer-to-peer networks
- Database consistency checking

Code :-

```
# Experiment 1

import hashlib

# SHA-256 Hash Generation

def sha256_hash(data):
```

```

    """
    Generates SHA-256 hash of the given input string
    """
    return hashlib.sha256(data.encode()).hexdigest()

# Hash Generation with Nonce
def hash_with_nonce(data, nonce):
    """
    Generates SHA-256 hash by combining data with nonce
    """
    combined_data = data + str(nonce)
    return sha256_hash(combined_data)

# Proof-of-Work (Mining Simulation)

def proof_of_work(data, difficulty):
    """
    Finds a nonce such that the hash has leading zeros
    based on the difficulty level
    """
    nonce = 0
    target = "0" * difficulty

    while True:
        hash_result = hash_with_nonce(data, nonce)
        if hash_result.startswith(target):
            return nonce, hash_result
        nonce += 1

# Merkle Tree Construction

def merkle_root(transactions):
    """
    Generates Merkle Root from a list of transactions
    """
    # Step 1: Hash all transactions
    current_level = [sha256_hash(tx) for tx in transactions]

    # Step 2: Build tree until one hash remains
    while len(current_level) > 1:

```

```

    next_level = []

    # If odd number of hashes, duplicate last hash
    if len(current_level) % 2 != 0:
        current_level.append(current_level[-1])

    # Hash pairs of nodes
    for i in range(0, len(current_level), 2):
        combined_hash = current_level[i] + current_level[i + 1]
        parent_hash = sha256_hash(combined_hash)
        next_level.append(parent_hash)

    current_level = next_level

    return current_level[0]

# Main program

print("===== TASK 1: SHA-256 HASH GENERATION =====")
input_data = input("Enter a string to hash: ")
hash_result = sha256_hash(input_data)
print("SHA-256 Hash:", hash_result)

print("\n===== TASK 2: HASH WITH NONCE =====")
nonce_value = int(input("Enter a nonce value: "))
hash_nonce = hash_with_nonce(input_data, nonce_value)
print("Hash with Nonce:", hash_nonce)

print("\n===== TASK 3: PROOF-OF-WORK =====")
difficulty = int(input("Enter difficulty level (e.g., 2, 3, 4): "))
nonce, pow_hash = proof_of_work(input_data, difficulty)
print("Nonce found:", nonce)
print("Valid Hash:", pow_hash)

print("\n===== TASK 4: MERKLE TREE =====")
num_tx = int(input("Enter number of transactions: "))
transactions = []

```

```
for i in range(num_tx):
    tx = input(f"Enter transaction {i + 1}: ")
    transactions.append(tx)

root = merkle_root(transactions)
print("Merkle Root:", root)

print("\n===== EXPERIMENT COMPLETED =====")
```

Output :-

```
*** ===== TASK 1: SHA-256 HASH GENERATION =====
Enter a string to hash: qwertykeyboard
SHA-256 Hash: 08222af35f7c79715c018c2ec3101a725f417fd2c7223eff2546fc031a89ff9e

===== TASK 2: HASH WITH NONCE =====
Enter a nonce value: 20012026
Hash with Nonce: 76f499975122f50eb25fd10a9dbfd8da46d9298c6a9c55988b1749327bf95533

===== TASK 3: PROOF-OF-WORK =====
Enter difficulty level (e.g., 2, 3, 4): 5
Nonce found: 1058567
Valid Hash: 0000006f4e33c7ab1c5b1e85e7ba169a80324333ba24f8ff1779a3b09466439a

===== TASK 4: MERKLE TREE =====
Enter number of transactions: 3
Enter transaction 1: 12
Enter transaction 2: 24
Enter transaction 3: 36
Merkle Root: 466cf447ec27f73e45fd90c092b97c2db6ce8f6bf8a10a0c23e8598726d75cac

===== EXPERIMENT COMPLETED =====
```

Conclusion :-

This experiment was performed on Google Colab. The concept of Nonce value, its purpose, how it contributes to the mining process, and the concept of Proof-of-Work was understood.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Exp 2

Aim :- Create a Blockchain using Python

Theory :-

Blockchain

A blockchain is a distributed and immutable digital ledger used to record data in a secure and transparent manner. Data is stored in the form of blocks, and each block is cryptographically linked to the previous block, forming a continuous chain. This linking mechanism ensures data integrity, because altering one block would require changing all subsequent blocks.

Blockchain operates without a central authority. Instead, multiple participants maintain copies of the ledger, making the system decentralized and trustworthy.

Block Creation

In a blockchain, new blocks are created using a predefined structure. Each block typically contains:

- **Index** – Position of the block in the chain
- **Timestamp** – Time when the block is created
- **Proof** – Result of the Proof-of-Work algorithm
- **Previous Hash** – Hash value of the previous block

The first block in the blockchain is called the genesis block. It is created manually and does not reference any previous block.

Proof-of-Work (PoW)

Proof-of-Work is a consensus mechanism used to validate and add new blocks to the blockchain.

In this mechanism:

- A computational puzzle must be solved to mine a block
- The puzzle involves finding a value (proof) that produces a hash with specific properties
- This process requires computational effort and time

Proof-of-Work prevents malicious users from easily modifying the blockchain and helps secure the network.

Hashing

Blockchain uses cryptographic hash functions, commonly SHA-256, to generate hashes.

Hashing has the following properties:

- Produces a fixed-length output
- Even a small change in input produces a completely different hash
- Hashes cannot be reversed to obtain original data

Each block stores the hash of the previous block, which ensures that all blocks are securely linked.

Chain Validation

Blockchain validation ensures the integrity of the entire chain.

Validation involves:

1. Verifying that each block correctly stores the hash of the previous block
2. Ensuring that the Proof-of-Work condition is satisfied for every block

If any block is tampered with, validation fails, and the blockchain is marked as invalid.

Flask Web API

A Flask-based web API can be used to interact with the blockchain through HTTP requests.

Common API endpoints include:

- `/mine_block` – Mines and adds a new block
- `/get_chain` – Displays the entire blockchain
- `/is_valid` – Verifies the validity of the blockchain

This allows users to interact with the blockchain using a browser or API client.

Code :-

```
# -----  
# Simple Blockchain using Python and Flask  
# -----  
  
# Required:  
# pip install flask==2.2.5  
  
# Imports  
import datetime    # Timestamps  
import hashlib     # SHA-256 hashing  
import json        # JSON encoding  
from flask import Flask, jsonify # API framework  
  
# -----  
# Blockchain Logic  
# -----  
  
class Blockchain:  
  
    def __init__(self):
```

```
self.chain = []
# Genesis block
self.create_block(proof=1, previous_hash='0')

def create_block(self, proof, previous_hash):
    # Create and add a block
    block = {
        'index': len(self.chain) + 1,
        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash
    }
    self.chain.append(block)
    return block

def get_previous_block(self):
    # Return last block
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    # Find proof with hash starting with '0000'
    new_proof = 1
    while True:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()
        if hash_operation[:4] == '0000':
            return new_proof
        new_proof += 1

def hash(self, block):
    # Hash a block
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()
```

```

def is_chain_valid(self, chain):
    # Check chain integrity
    previous_block = chain[0]
    block_index = 1

    while block_index < len(chain):
        block = chain[block_index]

        if block['previous_hash'] != self.hash(previous_block):
            return False

        hash_operation = hashlib.sha256(
            str(block['proof']**2 - previous_block['proof']**2).encode()
        ).hexdigest()

        if hash_operation[:4] != '0000':
            return False

        previous_block = block
        block_index += 1

    return True

# -----
# Flask API
# -----

app = Flask(__name__)
blockchain = Blockchain()

# Mine a block
@app.route('/mine_block', methods=['GET'])
def mine_block():

```

```
previous_block = blockchain.get_previous_block()
proof = blockchain.proof_of_work(previous_block['proof'])
previous_hash = blockchain.hash(previous_block)
block = blockchain.create_block(proof, previous_hash)
```

```
return jsonify({
    'message': 'Block mined',
    'index': block['index'],
    'timestamp': block['timestamp'],
    'proof': block['proof'],
    'previous_hash': block['previous_hash']
}), 200
```

Get blockchain

```
@app.route('/get_chain', methods=['GET'])
```

```
def get_chain():
```

```
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200
```

Validate blockchain

```
@app.route('/is_valid', methods=['GET'])
```

```
def is_valid():
```

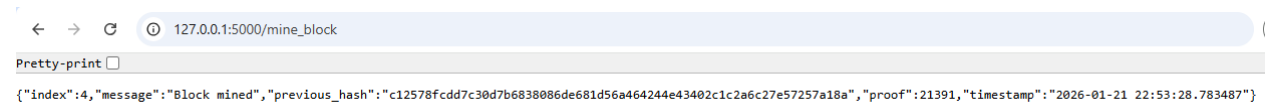
```
    if blockchain.is_chain_valid(blockchain.chain):
        return jsonify({'message': 'Blockchain is valid'}), 200
    return jsonify({'message': 'Blockchain is not valid'}), 200
```

Run server

```
app.run(host='0.0.0.0', port=5000)
```

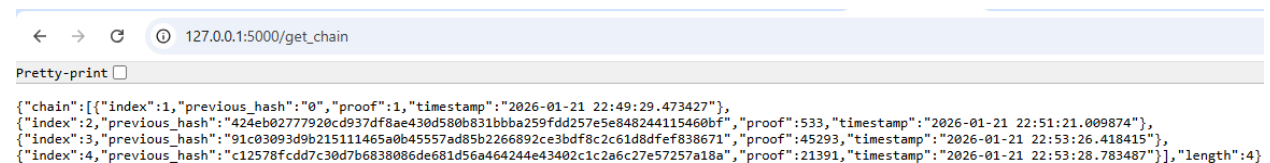
Output :-

Mine_block



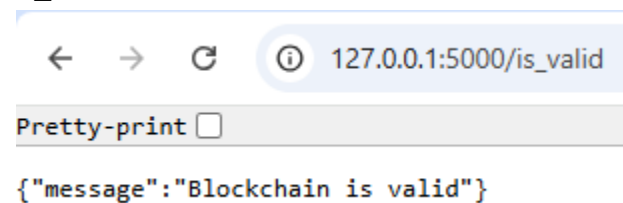
```
{\"index\":4,\"message\":\"Block mined\",\"previous_hash\":\"c12578fcdd7c30d7b6838086de681d56a464244e43402c1c2a6c27e57257a18a\",\"proof\":21391,\"timestamp\":\"2026-01-21 22:53:28.783487\"}
```

Get_chain



```
{\"chain\": [{\"index\":1,\"previous_hash\":\"0\",\"proof\":1,\"timestamp\":\"2026-01-21 22:49:29.473427\"}, {\"index\":2,\"previous_hash\":\"424eb02777920cd937df8ae430d580b831bbba259fdd257e5e848244115460bf\",\"proof\":533,\"timestamp\":\"2026-01-21 22:51:21.009874\"}, {\"index\":3,\"previous_hash\":\"91c03093d9b215111465a0b45557ad85b2266892ce3bdf8c2c61d8dfef838671\",\"proof\":45293,\"timestamp\":\"2026-01-21 22:53:26.418415\"}, {\"index\":4,\"previous_hash\":\"c12578fcdd7c30d7b6838086de681d56a464244e43402c1c2a6c27e57257a18a\",\"proof\":21391,\"timestamp\":\"2026-01-21 22:53:28.783487\"}], \"length\":4}
```

Is_valid



```
{\"message\":\"Blockchain is valid\"}
```

Conclusion :-

The experiment was performed locally on VSCode via Python 3. The output was displayed and user interaction was performed on 3 browser tabs, one for mining, one for viewing the chain, and one for checking validity of the block. The concept of mining and blockchain validity checking was understood via this experiment.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Exp 3

Aim :- Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory :- A

Blockchain Overview

A blockchain is a distributed, immutable ledger composed of blocks linked via cryptographic hashes. Each block contains a hash of the previous block, a timestamp, and transaction data—typically organized using a Merkle tree. This structure ensures that altering any block requires changing all subsequent blocks and achieving network-wide consensus, making tampering practically infeasible.

Blockchains operate on peer-to-peer (P2P) networks where nodes collectively maintain the ledger. They are categorized into public (permissionless), private, consortium, and hybrid types. Public blockchains like Bitcoin and Ethereum allow anyone to join, validate transactions, and participate in consensus without central authority.

Mining and Its Role

Mining is the process by which new blocks are created and added to the blockchain. In proof-of-work (PoW) systems like Bitcoin, miners compete to solve a computationally intensive cryptographic puzzle based on the SHA-256 algorithm. The first miner to find a valid solution—producing a hash below a target value—broadcasts the block to the network for validation.

Mining serves two critical purposes:

Securing the network against double-spending and fraud.

Introducing new cryptocurrency into circulation through block rewards.

The difficulty of the puzzle adjusts every 2,016 blocks (approximately every two weeks in Bitcoin) to maintain an average block time of 10 minutes.

Multi-Node Blockchain Network

A blockchain network consists of nodes, each maintaining a copy of the entire ledger. Nodes can be classified as:

Full nodes: Store the complete blockchain and validate transactions independently.

Light nodes: Store only block headers, relying on full nodes for data.

Mining nodes (miners): Participate in block creation and validation.

Super nodes: Relay information between nodes to ensure network synchronization.

All nodes enforce consensus rules, ensuring agreement on the state of the blockchain. When a new block is mined, it is propagated across the network, and each node verifies its validity before appending it to their local copy.

Consensus Mechanisms

Consensus mechanisms ensure all nodes agree on the validity of transactions and the state of the blockchain. The most common types include:

Proof of Work (PoW)

Used by Bitcoin, PoW requires miners to expend computational effort solving puzzles. The longest chain (representing the most work) is accepted as canonical. Security relies on the assumption that honest nodes control more than 50% of the network's computing power.

Proof of Stake (PoS)

Used by Ethereum post-"Merge", PoS selects validators based on the amount of cryptocurrency they "stake" as collateral. Validators propose and attest to blocks, earning rewards in return. This method is significantly more energy-efficient than PoW.

Other variants include Delegated Proof of Stake (DPoS), Practical Byzantine Fault Tolerance (PBFT), and hybrid models.

Transactions and Mining Reward

When a user initiates a transaction (e.g., sending bitcoin), it is broadcast to the network and held in a mempool (memory pool) until picked up by a miner. Miners bundle multiple transactions into a block and attempt to mine it.

Miners are incentivized through two types of rewards:

Block reward (coinbase reward): Newly minted cryptocurrency (e.g., currently 6.25 BTC per block in Bitcoin, halving approximately every four years).

Transaction fees: Paid by users to prioritize their transactions.

As block rewards decrease over time, transaction fees are expected to become the primary income source for miners. After 2140, when no new bitcoins will be created, mining will be sustained solely by transaction fees.

Chain Replacement and Fork Resolution

When two miners solve a block at nearly the same time, a temporary fork occurs, resulting in two competing chains. Nodes continue building on whichever chain they receive first. The conflict is resolved through the longest chain rule: the chain with the most accumulated proof-of-work becomes the accepted version.

Code :-

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof=1, previous_hash='0')
        self.nodes = set()
    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,

            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions
        }
        self.transactions = []
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while not check_proof:
            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof
```

```

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()

            ).hexdigest()
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
    return True

def add_transaction(self, sender, receiver, amount):
    self.transactions.append({
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    })
    previous_block = self.get_previous_block()
    return previous_block['index'] + 1

def add_node(self, address):
    parsed_url = urlparse(address)
    self.nodes.add(parsed_url.netloc)

def replace_chain(self):
    network = self.nodes
    longest_chain = None
    max_length = len(self.chain)
    for node in network:
        response = requests.get(f'http://{node}/get_chain')
        if response.status_code == 200:
            length = response.json()['length']
            chain = response.json()['chain']
            if length > max_length and self.is_chain_valid(chain):
                max_length = length
                longest_chain = chain
    if longest_chain:
        self.chain = longest_chain
        return True
    return False

app = Flask(__name__)

node_address = str(uuid4()).replace('-', '')

```

```

blockchain = Blockchain()

@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(
        sender=node_address,
        receiver='Richard',
        amount=1
    )
    block = blockchain.create_block(proof, previous_hash)
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }
    return jsonify(response), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }
    return jsonify(response), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'The Blockchain is not valid.'}

```

```

        return jsonify(response), 200

@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    json_data = request.get_json()
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json_data for key in transaction_keys):
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(
        json_data['sender'],
        json_data['receiver'],
        json_data['amount']
    )
    response = {
        'message': f'This transaction will be added to Block {index}'
    }
    return jsonify(response), 201

@app.route('/connect_node', methods=['POST'])
def connect_node():
    json_data = request.get_json()
    nodes = json_data.get('nodes')
    if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {
        'message': 'All the nodes are now connected.',
        'total_nodes': list(blockchain.nodes)
    }
    return jsonify(response), 201

@app.route('/replace_chain', methods=['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {
            'message': 'The chain was replaced by the longest one.',
            'new_chain': blockchain.chain
        }
    else:
        response = {
            'message': 'The chain is already the largest one.',
            'actual_chain': blockchain.chain
        }
    return jsonify(response), 200

app.run(host='0.0.0.0', port=5000)

```

Output :-

GET echo GET

Anchor tags in Postman documentation / GET Requests / echo GET

GET http://127.0.0.1:5000/mine_block

Overview Params Authorization Headers (7) Body Scripts Settings

Headers 7 hidden

Key	Value
Key	Value

Body Cookies Headers (5) Test Results

JSON Preview Visualize

```
1 {
2   "block": {
3     "index": 3,
4     "previous_hash": "4d56ca2f3e090aa136cbc8e9c40c5b4154438c7ed33a85fd9461e8c53d2afe84",
5     "proof": 45293,
6     "timestamp": "2026-02-03 09:29:02.261406",
7     "transactions": [
8       {
9         "amount": 1,
10        "receiver": "Miner",
11        "sender": "f50c937a472445029581f4c4dd5626fd"
12      }
13    ]
14  },
15  "message": "Block mined successfully"
16 }
```

GET echo GET

Anchor tags in Postman documentation / GET Requests / echo GET

GET http://127.0.0.1:5000/get_chain

Overview Params Authorization Headers (7) Body Scripts Settings

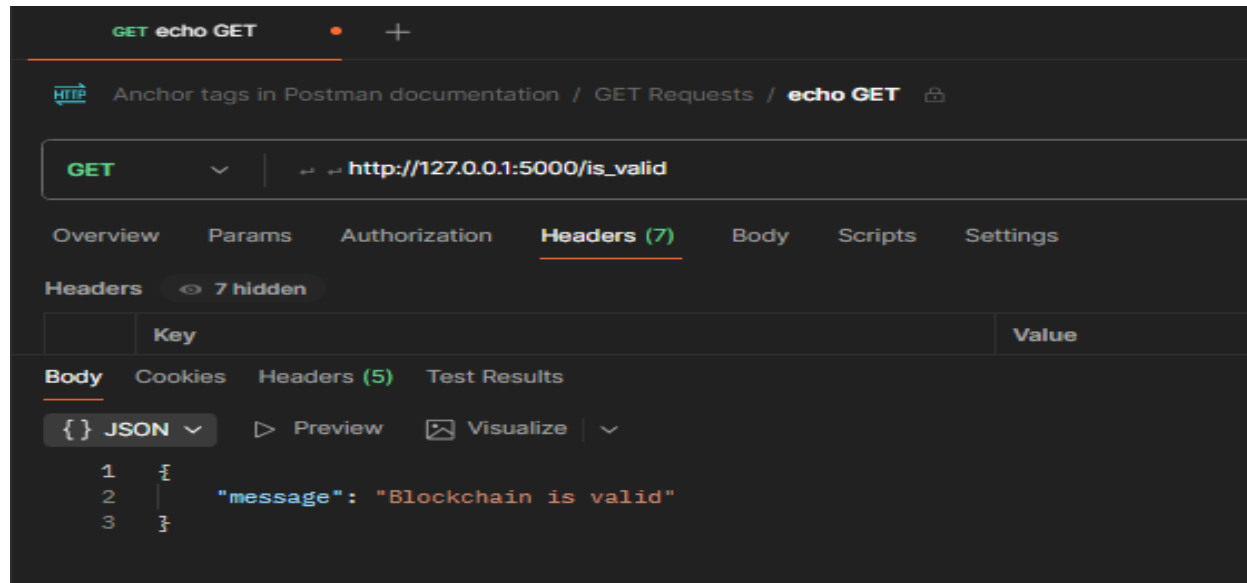
Headers 7 hidden

Key	Value
Key	Value

Body Cookies Headers (5) Test Results

JSON Preview Visualize

```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-02-03 09:19:02.441105",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "badca02819f0e8ae886535c28b2aef8affbd61def0f9537257cb84f22913acd7",
13      "proof": 533,
14      "timestamp": "2026-02-03 09:28:30.853839",
15      "transactions": [
16        {
17          "amount": 1,
18          "receiver": "Miner",
19          "sender": "f50c937a472445029581f4c4dd5626fd"
20        }
21      ]
22    }
23  ]
24 }
```



Conclusion :-

This implementation demonstrates how blockchain technology works by creating a decentralized ledger using Python. The code shows how blocks are securely linked using cryptographic hashes, how transactions are recorded, and how Proof-of-Work is used to mine new blocks. The use of consensus through chain validation and replacement ensures data integrity and synchronization across nodes. Overall, the system highlights the core principles of blockchain such as decentralization, transparency, security, and immutability.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Exp 4

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
 - o **public:** available both inside and outside the contract.
 - o **private:** only accessible within the same contract.
 - o **internal:** accessible within the contract and its child contracts.
 - o **external:** can be called only by external accounts or other contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

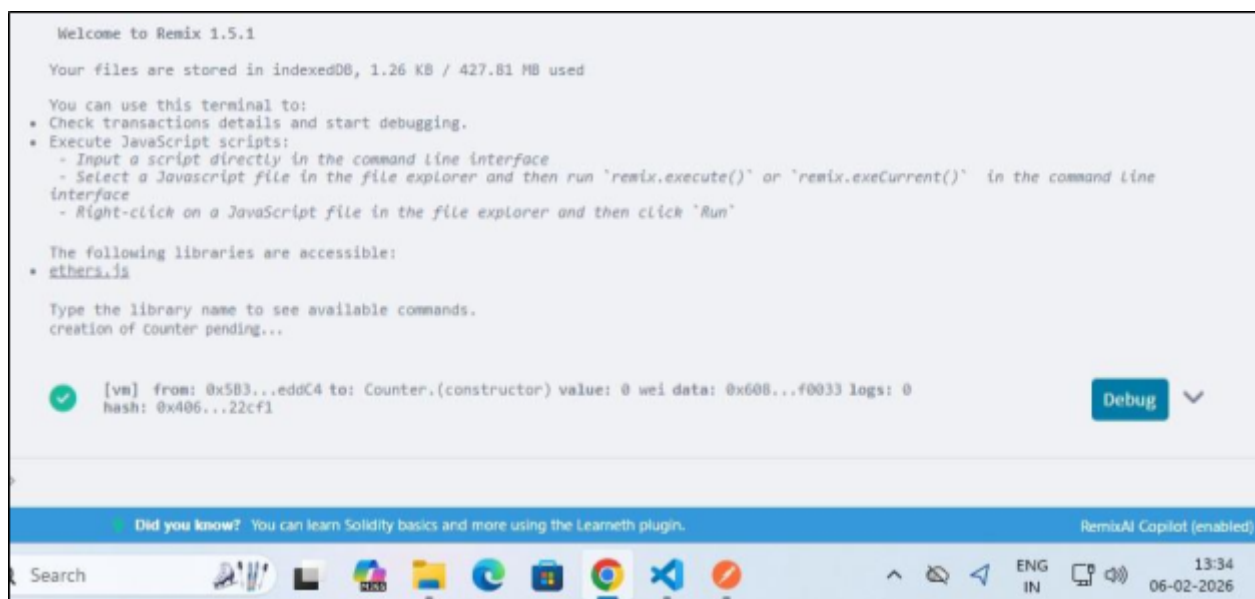
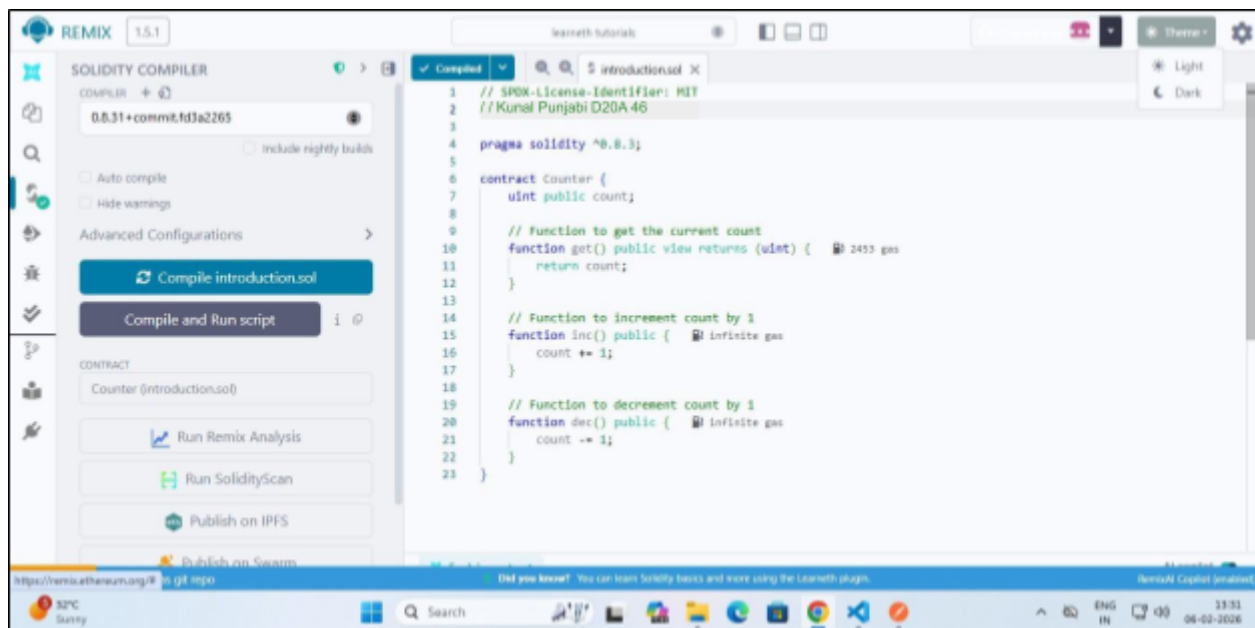
- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit ($1 \text{ Ether} = 10^{18} \text{ Wei}$). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

Tutorial - 1 (compile)



Tutorial - 1 (inc)

✓ COUNTER AT 0XF8E...9FBEB (MEMORY)

Balance: 0 ETH

dec

inc

count

get

0: uint256: 1

Low level interactions

i

CALLDATA

Transact

Tutorial - 1 (dec)

✓ COUNTER AT 0XF8E...9FBEB (MEMORY)

Balance: 0 ETH

dec

inc

count

get

0: uint256: 0

Low level interactions

i

CALLDATA

Transact

Tutorial - 2

LEARNETH

Tutorials list

2. Basic Syntax
2 / 19

Don't worry if you didn't understand some concepts like visibility, data types, or state variables. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

[Watch a video tutorial on Basic Syntax.](#)

★ **Assignment**

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

basicSyntax.sol

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4 // @naitual Pandey D20A 37
5 // Kunal Punjabi D20A 46
6 string public name = "Alice";
7 }
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Did you know? You can learn Solidity basics and more using the Learneth plugin.

RemixAI Copilot (enabled)

32°C Sunny

Search

ENG IN

13:46 06-02-2026

Tutorial - 3

★ **Assignment**

1. Create a new variable `newAddr` that is a `public address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

newContract.sol

```
26 address public addr = 0xCA35b7d915458Ef54Aa66088dF62F44E8fA733c;
27
28
29 // Kunal Punjabi D20A 46 37
30
31 address public newAddr = 0x0000000000000000000000000000000000000000;
32 int public neg = -37;
33 uint public newU = 0;
34
35
36 // Default values
37 // Unassigned variables have a default value
38 bool public defaultBool = false;
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Did you know? You can learn Solidity basics and more using the Learneth plugin.

RemixAI Copilot (enabled)

32°C Sunny

Search

ENG IN

13:54 06-02-2026

Tutorial - 4

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `getMessage()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check AnswerShow answer

Next

Well done! No errors.

```
10
11
12
13
14
15
16
17
18
19
20 }

uint public blockNumber;

function getMessage() public {
    // Here are some global variables
    uint timestamp = block.timestamp; // Current block timestamp
    address sender = msg.sender; // address of the caller
}
```

ⓘ Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

⌕

⌵

⌵

✕

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 5

★ Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check AnswerShow answer

Next

Well done! No errors.

```
18
19
20
21
22
23 }

bool public b = true;

function get_b() public view returns (bool) {
    return b;
}
```

ⓘ Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

⌕

⌵

⌵

✕

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 6

★ Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check AnswerShow answer

Next

Well done! No errors.

```
17
18
19
20 }

function addToX2(uint y) public {
    x = x + y;
}
```

ⓘ Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

⌕

⌵

⌵

✕

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 7

Watch a video tutorial on Function Modifiers.

★ Assignment

1. Create a new function, `increase` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increase` should be empty.

Tip: Use modifiers.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
55 modifier greatthan(uint y){
56     require(y > 0, "Input parameter not greater than 0");
57 }
58
59 modifier actualincrease(uint y){
60     _;
61     x = x + y;
62 }
63 // Kunal Punjabi D20A46
64 function increase(uint y) public onlyOwner greatthan(y) actualincrease(y){
65 }
66 }
```

[Explain contract](#) AI copilot

0 ☐ Listen on all transactions

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 8

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
80 function returnTwo() public pure returns(int p, bool q) {
81     p = -2;
82     q = true;
83 }
```

[Explain contract](#) AI copilot

0 ☐ Listen on all transactions

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 9

Create a new function in the `D011` contract called `testInternalVar` that returns the values of all state variables from the `base` contract that are possible to return.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
67 function testInternalVar() public view returns (string memory a, string memory b){
68     return (internalVar, publicVar);
69 }
```

[Explain contract](#) AI copilot

0 ☐ Listen on all transactions

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 10

★ Assignment

Create a new function called `evenCheck` in the `base` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```
18 // }
19 // return 2;
20
21 // shorthand way to write if / else statement
22 return _x < 10 ? 1 : 2;
23 }
24 // Kunal Punjabi D20A46
25 function evenCheck(uint a) public pure returns (bool){
26     return a % 2 == 0 ? true : false;
27 }
28 }
```

[Explain contract](#) AI copilot

0 ☐ Listen on all transactions

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 11

< Tutorials list Syllabus

7.2 Control Flow - Loops 11 / 19

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

[Watch a video tutorial on Loop statements.](#)

★ **Assignment**

1. Create a public `uint` state variable called `count` in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Loop {
5     //Kunal Punjabi D20A 46
6     uint public count;
7     function loop() public {
8         // for loop
9         for (uint i = 0; i < 10; i++) {
10             if (i == 5) {
11                 // Skip to next iteration with continue
12                 continue;
13             }
14             if (i == 5) {
15                 // Exit loop with break
16                 break;
17             }
18             count++;
19         }
20     }
21     // while loop
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

note: the called function should be payable if you send value and the value you send should be less than your current b...
if the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 12

< Tutorials list Syllabus

8.1 Data Structures - Arrays 12 / 19

important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

[Watch a video tutorial on Arrays.](#)

★ **Assignment**

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1,
2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer Show answer

Next

Well done! No errors.

```
7     uint[] public arr2 = [1, 2, 3];
8     //Kunal Punjabi D20A 46 D20A 37
9     uint[3] public arr3 = [0, 1, 2];
10    // Fixed sized array, all elements initialize to 0
11    uint[10] public myFixedSizeArr;
12
13    function get(uint i) public view returns (uint) {
14        return arr[i];
15    }
16
17    // Solidity can return the entire array.
18    // But this function should be avoided for
19    // arrays that can grow indefinitely in length.
20    function getArr() public view returns (uint[3] memory) {
21        return arr3;
22    }
23
24    function push(uint i) public {
25        // Append to array
26        // This will increase the array length by 1.
27        arr2.push(i);
28    }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

note: the called function should be payable if you send value and the value you send should be less than your current b...
if the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 13

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

[Watch a video tutorial on Mappings.](#)

★ Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check AnswerShow answer

Next

Well done! No errors.

```
6 mapping(address => uint) public myMap;
7 // Kunal Punjabi D20A46e9 D20A 37
8 mapping(address => uint) public balances;
9
10 function get(address _addr) public view returns (uint) { 2895 gas
11     // Mapping always returns a value.
12     // If the value was never set, it will return the default value.
13     return balances[_addr];
14 }
15
16 function set(address _addr) public { 25279 gas
17     // Update the value at this address
18     balances[_addr] = _addr.balance;
19 }
20
21 function remove(address _addr) public { 5588 gas
```

[Explain contract](#)

0 ☐ Listen on all transactions

Note: The called function should be payable if you send value and the value you send should be less than you. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 14

[Watch a video tutorial on Structs.](#)

★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check AnswerShow answer

Next

Well done! No errors.

```
46 }
47 // Kunal Punjabi D20A46
48 function remove(uint _index) public { infinite gas
49     delete todos[_index];
50 }
```

[Explain contract](#)

0 ☐ Listen on all transactions

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial 15

15 / 19

Another way to update the value is using the `delete` operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `size` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `size`.

Check AnswerShow answer

Next

Well done! No errors.

```
15
16
17 L
18 }
19 // Default value is the first element listed in
20 // definition of the type, in this case "Pending"
21 Status public status;
22 Size public size;
23 // Returns uint
24 // Pending - 0
25 // Shipped - 1
26 // Accepted - 2
27 // Rejected - 3
28 // Canceled - 4
29 function getSize() public view returns(Size){ 2655 gas
30     return size;
31 }
```

[Explain contract](#)

0 ☐ Listen on all transactions

Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tutorial - 16

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myNewStruct2` with the data location `memory` inside the `function f` and assign it the value of `myNewStruct`. Change the value of the `myNewStruct2` member `foo` to 1.
3. Create a new struct `myNewStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myNewStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myNewStruct2`, and `myNewStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check AnswerShow answer

Next

Well done! No errors.

```
25 return (myStruct, myNewStruct2, myNewStruct3);
26
27
28 function f(    ) undefined gas
29     uint[] storage _arr,
30     mapping(uint => address) storage _map,
31     MyStruct storage _myStruct
32 ) internal {
33     // do something with storage variables
34 }
35
36 // You can return memory variables
37 function g(uint[] memory _arr) public returns (uint[] memory) {    infinite gas
38     // do something with memory array
39     _arr[0] = 1;
40 }
41 //Prajjwal Pandey D20A 37
42 //Kunal Punjabi D20A 46 calldata _arr) external {    468 gas
43     // do something with calldata array
44     // _arr[0] = 1;
45 }
46 }
```

Activate Windows
Go to Settings to activate Windows.

Tutorial 17

< Tutorials list

Syllabus

10.1 Transactions - Ether and Wei

17 / 19

gwei

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

ether

One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).

[Watch a video tutorial on Ether and Wei.](#)

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check AnswerShow answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
17 // Kunal Punjabi D20A 46
```

Tutorial 18

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin Gas Profiler to check for the gas cost of transactions.

Code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7     //Kunal Punjabi D20A 46/ D20A 37
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

Well done! No errors.

Tutorial 19

Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Code:

```
43 }
44
45 function sendViaCall(address payable _to) public payable {
46     // Call returns a boolean value indicating success or failure.
47     // This is the current recommended method to use.
48     (bool sent, bytes memory data) = _to.call{value: msg.value}("");
49     require(sent, "Failed to send Ether");
50 }
51
52 //Pratishat Randev D20A 31
53 //Kunal Punjabi D20A 46
54 address public owner;
55
56 constructor() {
57     owner = msg.sender;
58 }
59
60 function donate() public payable {
61 }
62
63 function withdraw() public {
64     uint amount = address(this).balance;
65     (bool sent, bytes memory data) = owner.call{value: amount}("");
66     require(sent, "Failed to send Ether");
67 }
68 }
```

Well done! No errors.

Conclusion :

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

Exp 5

Aim: Deploying a Voting/Ballot Smart Contract

Theory:

1. Importance of 'require' Statements in Solidity

In Solidity, the require statement is used to validate conditions before executing critical parts of a smart contract. It acts as a safety checkpoint that ensures only legitimate inputs and authorized users can proceed with a function call. If the specified condition evaluates to false, the transaction is immediately reverted, and any changes made during execution are undone. This protects the contract from invalid operations and preserves blockchain integrity.

In a Voting (Ballot) smart contract, require statements can be applied to enforce rules such as:

- Verifying that a voter has the right to vote.
- Preventing a voter from casting multiple votes.
- Restricting certain functions (like granting voting rights) to the chairperson only.

Additionally, require allows developers to include descriptive error messages, which improve debugging and user interaction. Overall, it strengthens contract security, reliability, and correctness.

2. Key Solidity Concepts: mapping, storage, and memory mapping

A mapping in Solidity is a key-value data structure used to associate one type of data with another. Its syntax is:

mapping(keyType => valueType) For example:

```
mapping(address => Voter) public voters;
```

In this case, each Ethereum address is linked to a Voter struct containing relevant information such as voting weight, voting status, and selected proposal.

Mappings are highly efficient for data retrieval and are widely used in voting contracts. However, they do not maintain a length property and cannot be directly iterated over, making them efficient for lookups but limited for enumeration.

Storage

Storage refers to the permanent data area on the blockchain. State variables declared in a contract are stored in storage by default. Data stored here remains available across multiple transactions unless modified.

Because storage writes consume significant gas, developers must use it carefully. For example, voter details stored in a mapping remain permanently saved throughout the contract's lifecycle.

Memory

Memory is temporary storage used during function execution. Variables declared with the memory keyword exist only for the duration of the function call and are discarded afterward.

Memory is less expensive compared to storage and is suitable for temporary variables, calculations, or function parameters. For instance, temporary string handling or intermediate computations are typically done in memory.

Smart contract developers must carefully choose between storage and memory to optimize performance and reduce gas costs.

3. Why Use bytes32 Instead of string?

In earlier versions of the Ballot contract, proposal names were often stored as bytes32 instead of string.

- bytes32 is a fixed-size data type that stores exactly 32 bytes.
- It is gas-efficient, easier to compare, and simpler for the EVM to handle.
- However, it limits text length to 32 characters, reducing flexibility.

On the other hand:

- string is a dynamic data type that allows variable-length text.
- It improves readability and user-friendliness.
- However, it requires more complex storage handling and consumes more gas.

Therefore, bytes32 is preferred when efficiency and lower gas costs are priorities, while string is chosen when readability and flexibility are more important.

Code:

```
// SPDX-License-Identifier: GPL-3.0 pragma solidity ^0.8.0;
```

```
/**  
 * @title Ballot  
 * @dev Implements voting process along with vote delegation  
 */
```

```

contract Ballot {

    struct Voter {
        uint256 weight;          // weight is accumulated by delegation bool voted; // if true, that person
        already voted address delegate; // person delegated to
        uint256 vote;            // index of the voted proposal
    }

    struct Proposal {
        string name;             // proposal name
        uint256 voteCount; // number of accumulated votes
    }

    address public chairperson; mapping(address => Voter) public voters; Proposal[] public
    proposals;

    /**
     * @dev Create a new ballot to choose one of 'proposalNames'.
     */
    constructor(string[] memory proposalNames) { chairperson = msg.sender;
    voters[chairperson].weight = 1;

    for (uint256 i = 0; i < proposalNames.length; i++) { proposals.push(
    Proposal({
    name: proposalNames[i], voteCount: 0
    })
    );
    }
    }

    /**
     * @dev Give 'voter' the right to vote. Only chairperson can call.
     */
    function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right to vote");
    require(!voters[voter].voted, "The voter already voted"); require(voters[voter].weight == 0, "Voter
    already has voting rights");

    voters[voter].weight = 1

    }

    /**
     * @dev Delegate your vote to another voter.
     */
    function delegate(address to) external { Voter storage sender = voters[msg.sender];

    require(sender.weight > 0, "You have no right to vote"); require(!sender.voted, "You already
    voted");
    require(to != msg.sender, "Self-delegation is not allowed");

```

```

// Follow the chain of delegation
while (voters[to].delegate != address(0)) { to = voters[to].delegate;
require(to != msg.sender, "Delegation loop detected");
}

Voter storage delegate_ = voters[to];
require(delegate_.weight > 0, "Delegate has no right to vote");

sender.voted = true; sender.delegate = to;

if (delegate_.voted) {
// If the delegate already voted, add directly proposals[delegate_.vote].voteCount +=
sender.weight;
} else {
// If the delegate did not vote yet, add weight delegate_.weight += sender.weight;
}
}

/**
 * @dev Cast your vote.
 */
function vote(uint256 proposal) external { Voter storage sender = voters[msg.sender];

require(sender.weight > 0, "No right to vote"); require(!sender.voted, "Already voted");
require(proposal < proposals.length, "Invalid proposal index"); sender.voted = true

sender.vote = proposal; proposals[proposal].voteCount += sender.weight;

}

/**

 * @dev Returns index of winning proposal.

 */

function winningProposal() public view returns (uint256 winningProposal_) { uint256

winningVoteCount = 0;

for (uint256 p = 0; p < proposals.length; p++) {

if (proposals[p].voteCount > winningVoteCount) { winningVoteCount = proposals[p].voteCount;

winningProposal_ = p;

}

}

}

```

```
/**
```

```
* @dev Returns name of winning proposal.
```

```
*/
```

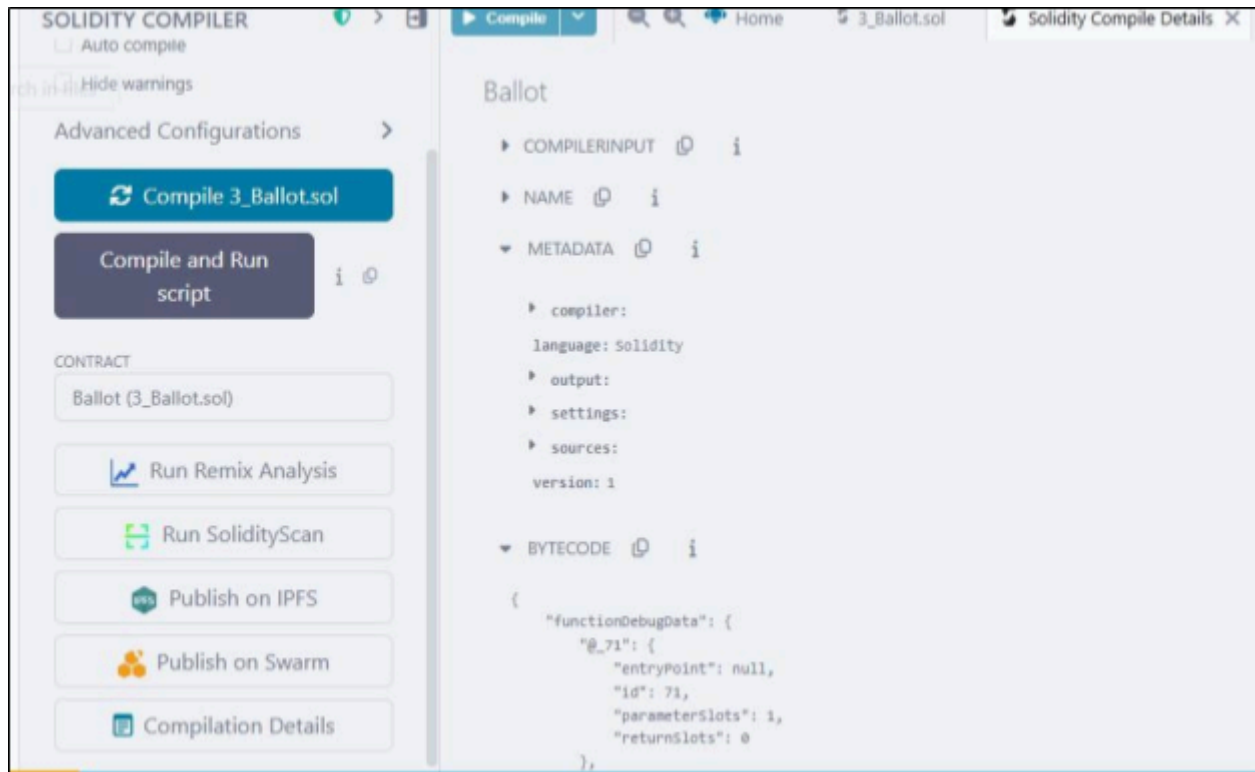
```
function winnerName() external view returns (string memory winnerName_) { winnerName_ =  
proposals[winningProposal()].name;
```

```
}
```

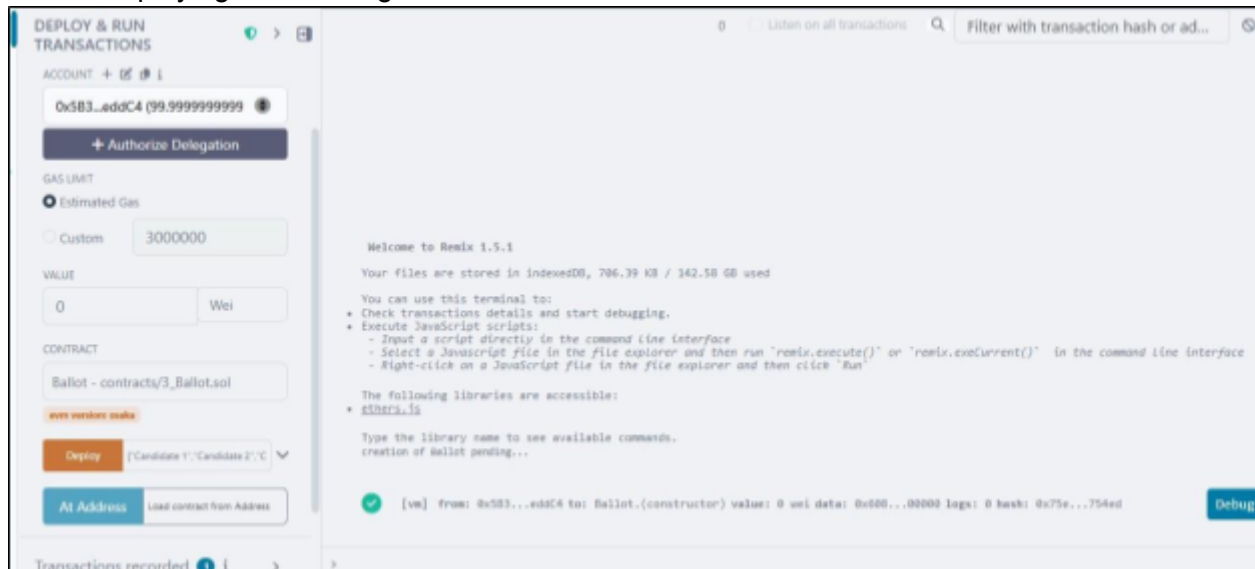
```
}
```

Output:

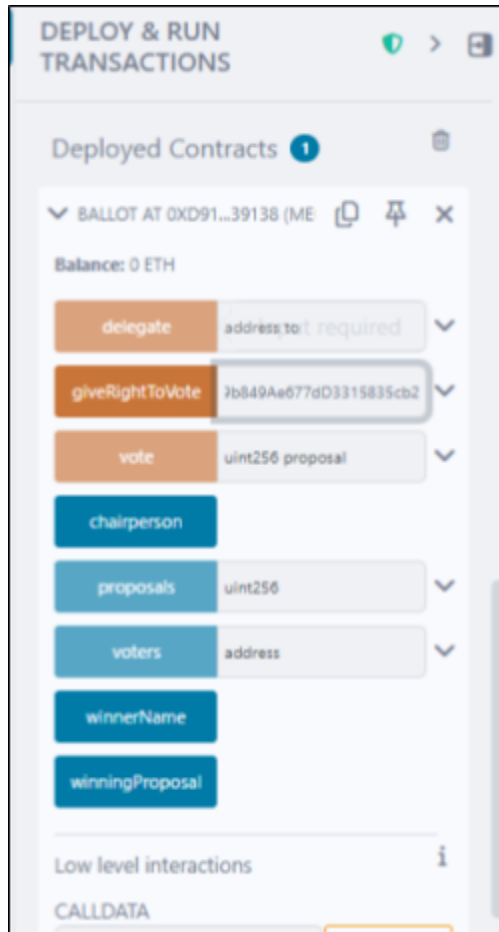
- Compiled Ballot.sol Contract



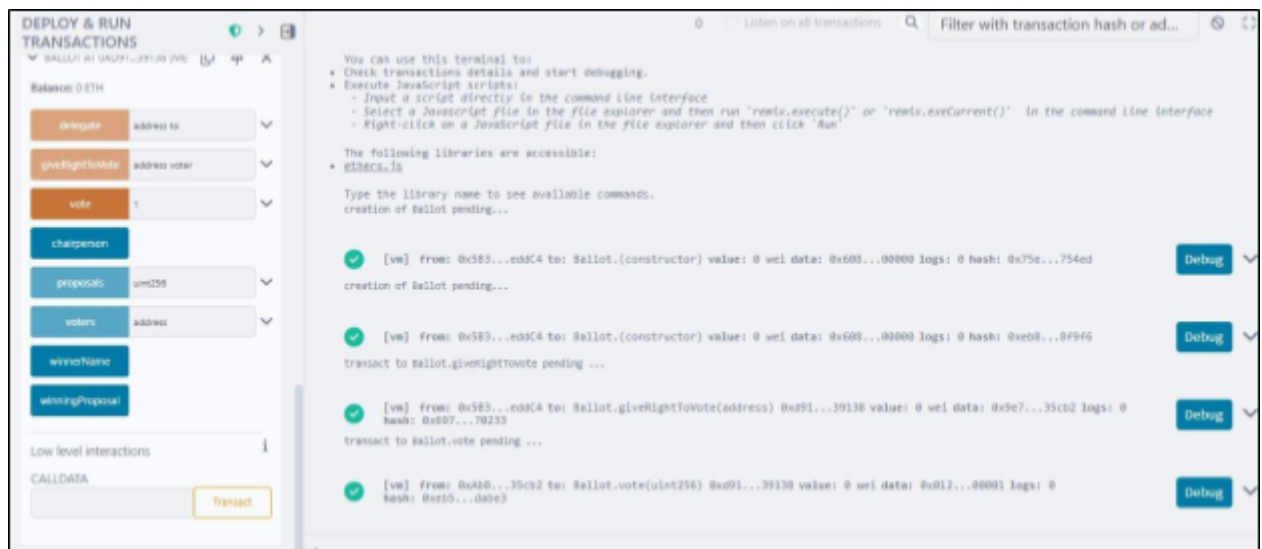
- Deploying and running of the contract



- Loading the Proposal Candidate's Names (string)



Voted successfully:



DEPLOY & RUN TRANSACTIONS

Deployed Contracts 1

BALLOT AT 0xD91...39138 (ME)

Balance: 0 ETH

delegate address to

giveRightToVote address voter

vote 1

chairperson

proposals uint256

voters address

winnerName

winningProposal

Low level interactions

0 Listen on all transactions Filter with transaction h

[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0x75e...754ed
creation of \$ballot pending...

[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0xeb5...8f9f6
transact to \$ballot.giveRightToVote pending ...

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xd91...39138 value: 0 wei data: 0x9e7...35cb2 logs:
hash: 0x697...70233
transact to \$ballot.vote pending ...

[vm] from: 0xab5...35cb2 to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...00001 logs: 0
hash: 0xeb5...dabe3
transact to \$ballot.vote pending ...

[vm] from: 0x4b2...C82db to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...00002 logs: 0
hash: 0x78d...5467f
transact to \$ballot.vote errored: Error occurred: revert.
revert
The transaction has been reverted to the initial state.
Reason provided by the contract: "No right to vote".
If the transaction failed for not having enough gas, try increasing the gas limit gently.

Tried to Vote twice which gave an error:

Deployed Contracts 1

BALLOT AT 0xD91...39138 (ME)

Balance: 0 ETH

delegate address to

giveRightToVote 0xd17f2e2f072f0d055031

vote 1

chairperson

proposals uint256 out of range

voters address

winnerName

0: string: winnerName_Candidate 2

winningProposal

Low level interactions

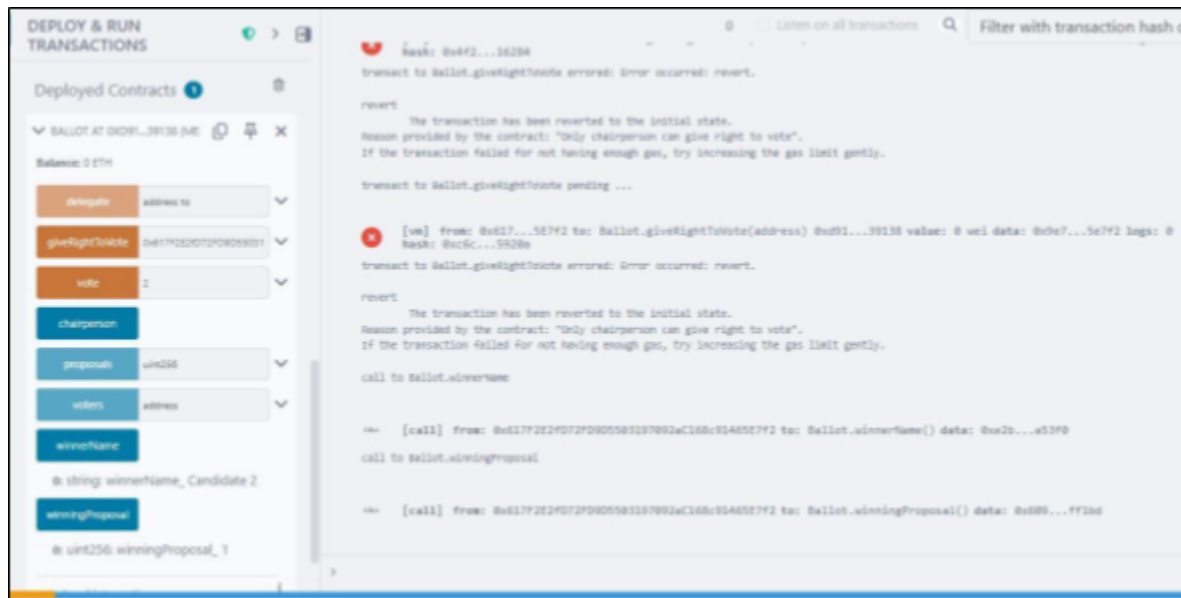
transact to \$ballot.giveRightToVote pending ...

[vm] from: 0x617...5E7f2 to: Ballot.giveRightToVote(address) 0xd91...39138 value: 0 wei data: 0x9e7...5e7f2 logs: 0
hash: 0x4f2...36264
transact to \$ballot.giveRightToVote errored: Error occurred: revert.
revert
The transaction has been reverted to the initial state.
Reason provided by the contract: "Only chairperson can give right to vote".
If the transaction failed for not having enough gas, try increasing the gas limit gently.
transact to \$ballot.giveRightToVote pending ...

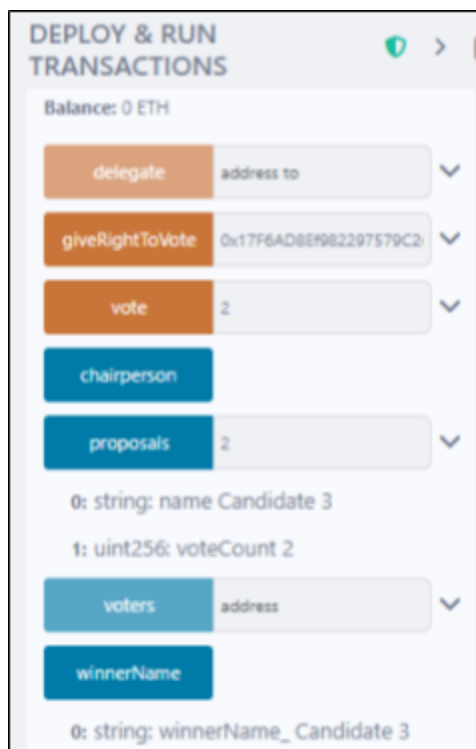
[vm] from: 0x617...5E7f2 to: Ballot.giveRightToVote(address) 0xd91...39138 value: 0 wei data: 0x9e7...5e7f2 logs: 0
hash: 0xc6c...5920a
transact to \$ballot.giveRightToVote errored: Error occurred: revert.
revert
The transaction has been reverted to the initial state.
Reason provided by the contract: "Only chairperson can give right to vote".
If the transaction failed for not having enough gas, try increasing the gas limit gently.
call to \$ballot.winnerName

[call] from: 0x617f2e2f072f0d05503197892aC168c91465E7f2 to: Ballot.winnerName() data: 0xe2b...a53f8

Voted from another node:



Checked the Winner node and the Proposals:



Conclusion

In this experiment, a Voting (Ballot) smart contract was implemented and deployed using Solidity in the Remix IDE environment. The practical use of 'require' statements demonstrated how smart contracts enforce validation rules and prevent unauthorized or incorrect actions. Core Solidity concepts such as mapping, storage, and memory were examined to understand how data is managed efficiently on the blockchain.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Exp 6

Aim :- Creating Smart Contracts and performing transactions using Solidity and Remix IDE.

Theory :-

1. Core Theory of Smart Contracts

What is a Smart Contract?

A smart contract is a self-executing program stored on a blockchain.

It automatically enforces rules and executes actions when predefined conditions are met.

Key characteristics:

- Immutable once deployed — code cannot be changed.
- Distributed — every node stores and verifies the contract.
- Deterministic — same input always produces the same output.
- Transparent — code and state are visible on-chain.

Smart contracts were conceptualized by Nick Szabo (1994) and became mainstream with Ethereum, which introduced a programmable blockchain.

2. Solidity: The Language of Ethereum Smart Contracts

Solidity is:

- Contract-oriented (everything revolves around contracts)
- Statically typed
- Compiled into EVM bytecode
- Influenced by JavaScript, C++, and Python

Key Solidity Concepts

Concept	Meaning
pragma	Specifies compiler version
state variables	Stored permanently on blockchain
functions	Define behavior (public, private, view, pure, payable)
events	Emit logs for off-chain listeners

modifiers	Reusable access-control logic
constructor	Runs once during deployment
mappings/structs	Data structures for storage
msg.sender / msg.value	Transaction metadata

3. Remix IDE: The Browser-Based Ethereum Development Environment

Remix is a web IDE that allows you to:

- Write Solidity code
- Compile contracts
- Deploy to a virtual blockchain (JavaScript VM)
- Deploy to testnets/mainnet via MetaMask
- Interact with deployed contracts
- Debug transactions

It removes the need for local setup and is ideal for beginners and rapid prototyping.

4. The Smart Contract Development Lifecycle

Step 1: Writing the Contract

You create a `.sol` file and define your contract.

Step 2: Compiling the Contract

Using Remix:

- Open Solidity Compiler
- Select version matching `pragma`
- Click Compile

Remix checks:

- Syntax errors
 - Type mismatches
 - Security warnings
-

Step 3: Deploying the Contract

Using Deploy & Run Transactions panel:

- Choose an environment (JavaScript VM, Injected Provider, etc.)

- Select contract
- Click Deploy

Deployment creates:

- A transaction that stores the contract bytecode on-chain
 - A contract address where the contract lives permanently
-

5. Performing Transactions with Smart Contracts

Once deployed, you can interact with the contract.

Types of Transactions

Type	Description
Call (free)	Read-only, no gas, doesn't change blockchain state
Transaction (costs gas)	Writes data, triggers state changes

How Remix Handles Transactions

- Each function appears as a button in the UI.
 - Calling a function sends a transaction to the contract.
 - Remix shows:
 - Gas used
 - Logs/events
 - Return values
 - State changes
-

6. Payable Functions & Ether Transfers

To send Ether:

```
function deposit() public payable { }
```

To withdraw:

```
function withdraw(uint amount) public {  
    payable(msg.sender).transfer(amount);  
}
```

Remix allows you to specify value (in wei/ether) before calling payable functions.

7. Testing & Debugging in Remix

Remix includes:

- Debugger (step through transactions)
- Console logs
- Gas profiler
- Static analysis tools

These help catch:

- Reentrancy vulnerabilities
 - Integer overflows
 - Uninitialized storage
 - Access control issues
-

8. Why Remix IDE Matters

According to Startupik, Remix lowers the barrier to entry by providing a browser-based environment where developers can write, compile, test, and deploy without complex setup.

This makes it ideal for:

- Students
 - Hackathon participants
 - Blockchain beginners
 - Rapid prototyping
-

9. Summary Table

Topic	Key Idea
Smart Contracts	Self-executing blockchain programs
Solidity	Main language for Ethereum contracts
Remix IDE	Browser-based IDE for writing & deploying contracts
Transactions	Calls (free) vs. state-changing (gas)
Deployment	Stores contract on blockchain with unique address
Ether Transfers	Use payable functions
Debugging	Built-in tools for analysis and testing

Code :-

AcademicRecords.sol

```
// SPDX-License-Identifier: MIT pragma solidity 0.8.19; contract AcademicRecords {

    struct Student { string name; string course; uint marks; string ipfsHash; bool exists;
    }
    mapping(string => Student) public students; function addStudent(
    string memory _id, string memory _name, string memory _course, uint _marks,
    string memory _ipfsHash
    ) public {
        students[_id] = Student(_name, _course, _marks, _ipfsHash, true);
    }

    function getStudent(string memory _id) public view
    returns (string memory, string memory, uint, string memory)
    {
        require(students[_id].exists, "Student not found"); Student memory s = students[_id];
        return (s.name, s.course, s.marks, s.ipfsHash);
    }
}
```

Verification.sol

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.0;

contract Verification {

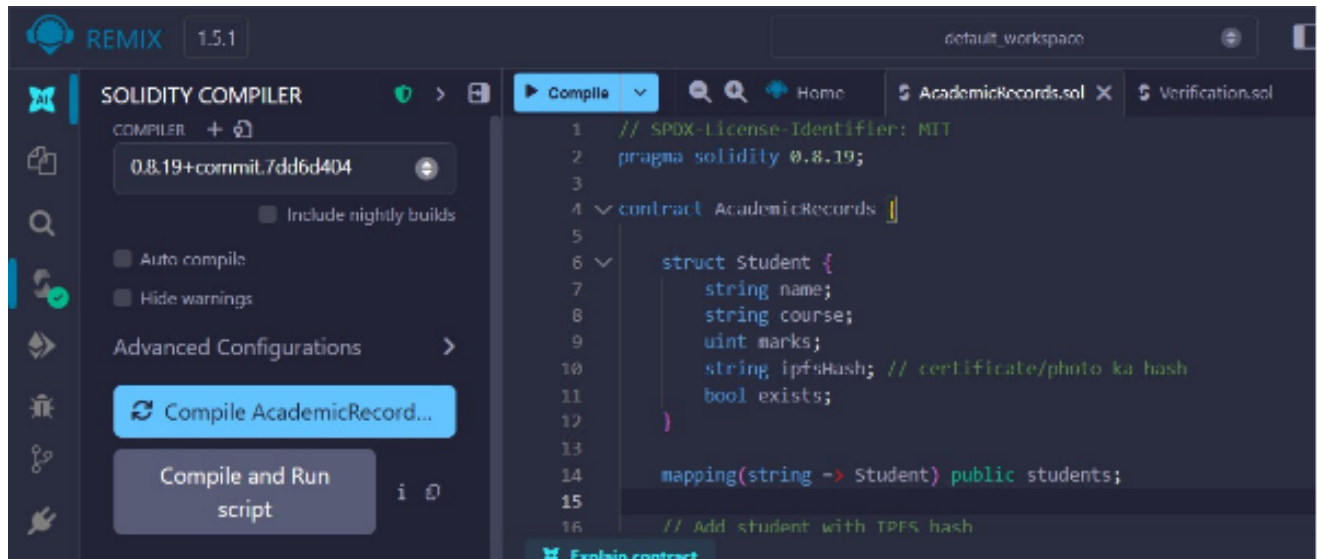
    mapping(string => bool) public verified;

    function verifyCertificate(string memory _id) public { verified[_id] = true;
    }

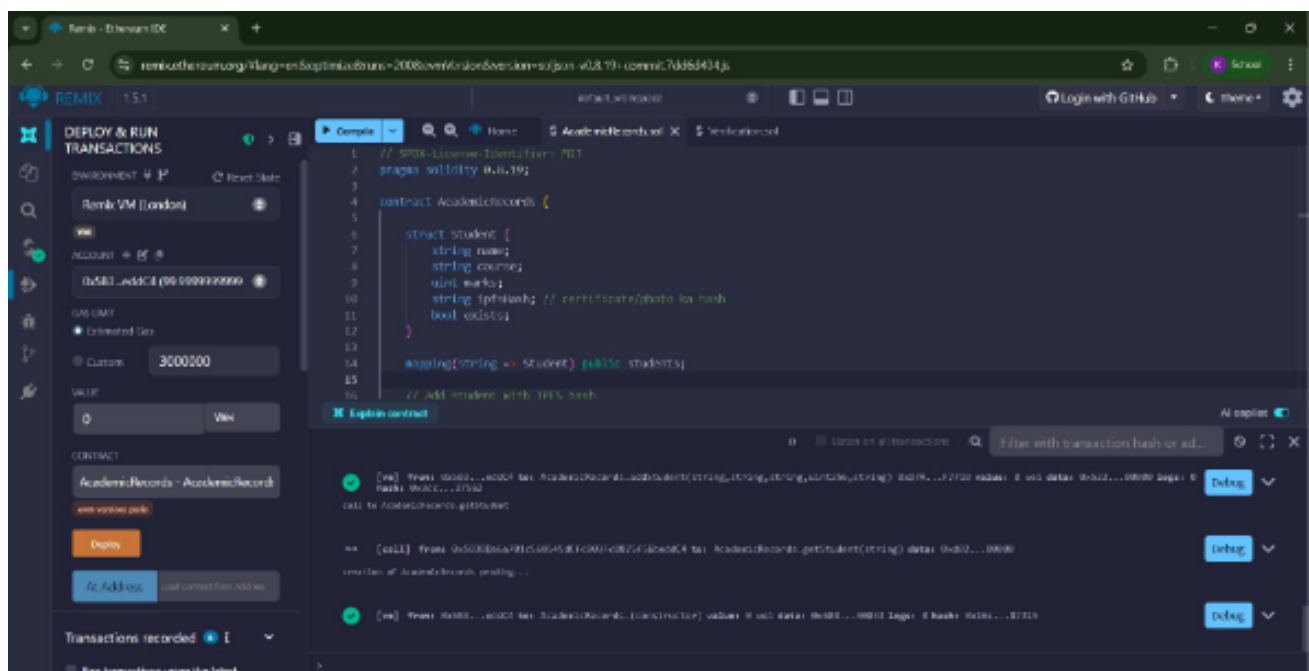
    function checkVerification(string memory _id) public view
    returns (bool)
    {
        return verified[_id];
    }
}
```

Screenshots :-

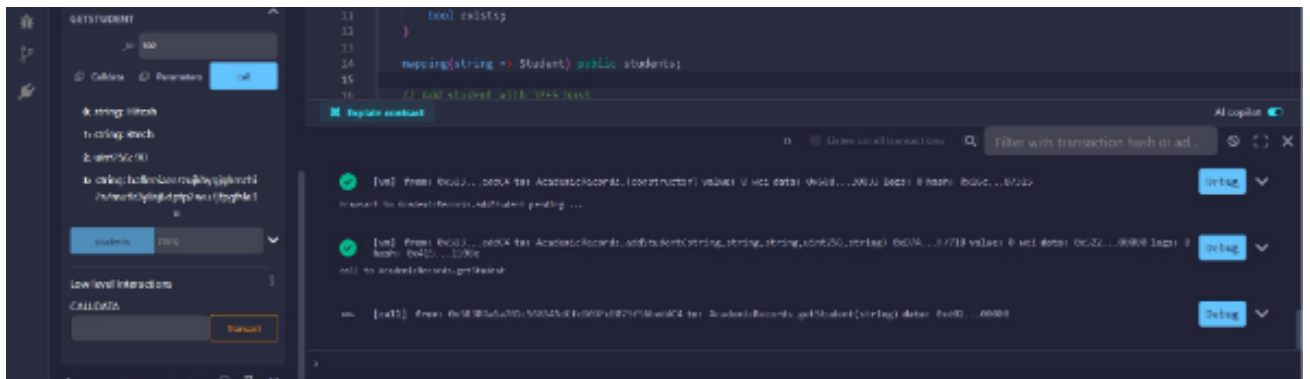
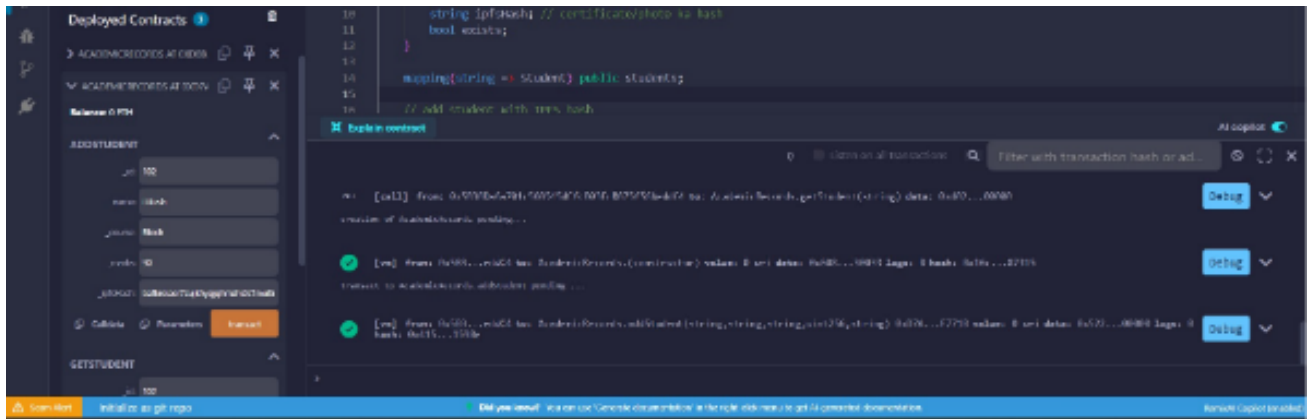
Compiling code :



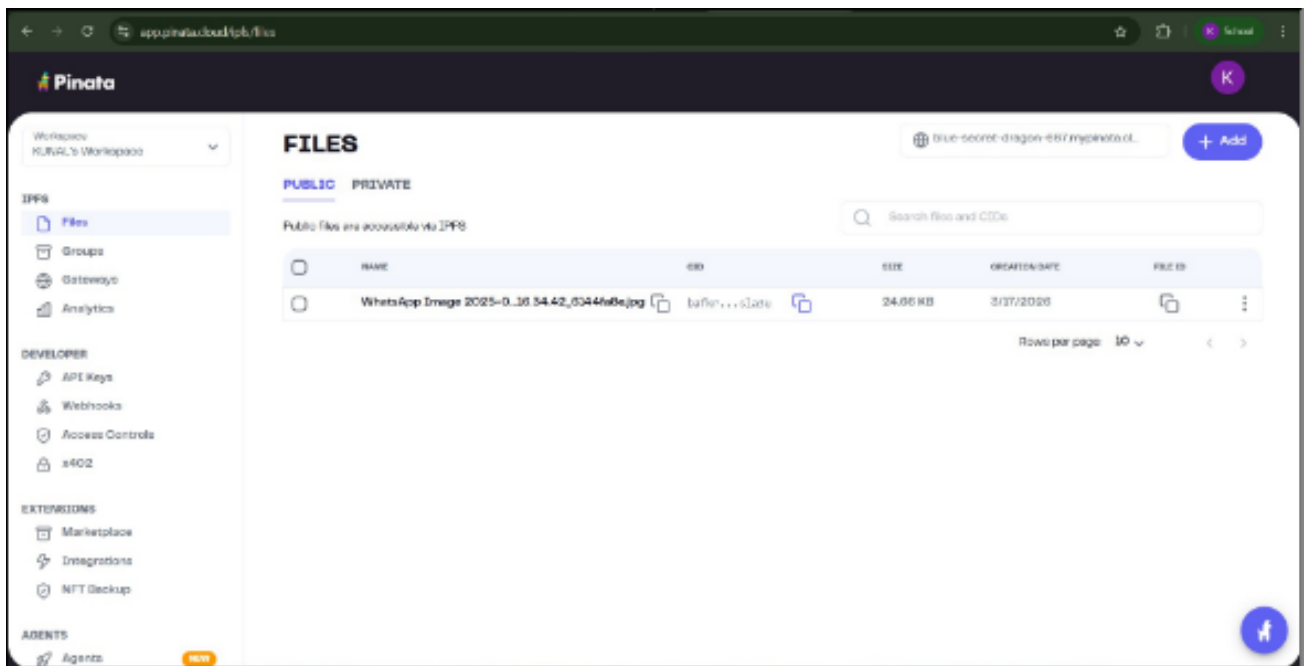
Deploying contract :



Contract execution :



Integration :



Conclusion :-

This experiment helped in understanding the practical implementation of smart contracts using Solidity in Remix IDE. A smart contract was successfully designed to store and retrieve student academic records on the blockchain, ensuring data security and immutability. The system was further enhanced by integrating IPFS to store certificate images, with only the hash stored on the blockchain. This approach makes the system efficient, secure, and resistant to data tampering. Overall, the experiment demonstrated how blockchain technology can be effectively used for reliable academic credential verification.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Exp 7

Aim :- To implement MetaMask wallet integration and perform transactions using Solidity.

Theory :-

MetaMask :

MetaMask is a cryptocurrency wallet that allows users to interact with blockchain networks. It acts as a bridge between the browser and blockchain, enabling transactions and smart contract interaction.

Testnet :

A testnet is a blockchain network used for testing purposes. It uses fake cryptocurrency like Sepolia ETH, allowing developers to test smart contracts without real money.

Academic Credential Verification using Blockchain :

In this system, student academic details such as name, course, and grade are stored on the blockchain using a smart contract. This ensures data is secure, tamper-proof, and easily verifiable.

Advantages :

- Secure and tamper-proof records
- Easy verification
- No dependency on centralized authority
- Transparent system

Steps to Connect MetaMask with Remix IDE :

1. Install MetaMask extension
2. Create a wallet account
3. Select Sepolia Testnet
4. Get free ETH from faucet
5. Open Remix IDE
6. Select "Injected Provider - MetaMask"
7. Connect MetaMask
8. Compile smart contract
9. Deploy contract
10. Confirm transaction
11. Interact with functions

Code :-

Program (Smart Contract) :

```
{
// SPDX-License-Identifier: MIT pragma solidity ^0.8.0;
contract AcademicCredentials { struct Credential {
    uint256 id;
    string studentName; string course;
    string grade; address issuedBy; uint256 timestamp;
}
mapping(uint256 => Credential) public credentials; event CredentialAdded(
uint256 id,
string studentName, string course,
string grade, address issuedBy, uint256 timestamp
);

function addCredential( uint256 _id,
string memory _studentName, string memory _course,

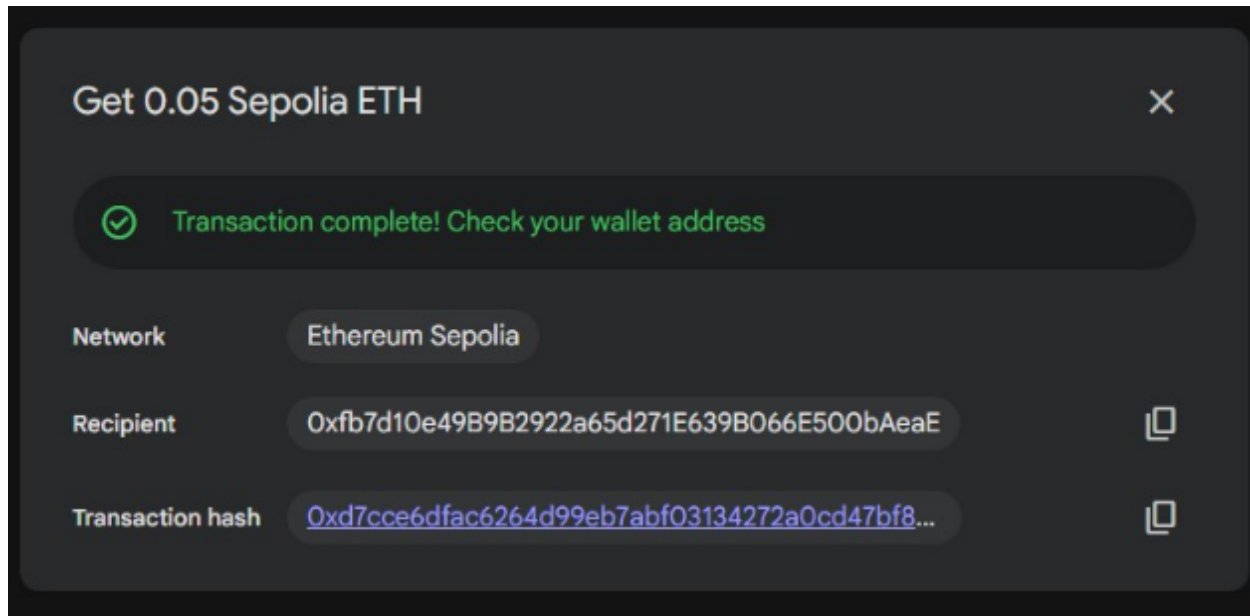
string memory _grade
) public {
    require(credentials[_id].id == 0, "Already exists!"); credentials[_id] = Credential(_id,
_studentName, _course, _grade, msg.sender, block.timestamp);

    emit CredentialAdded(_id, _studentName, _course, _grade, msg.sender, block.timestamp);
}

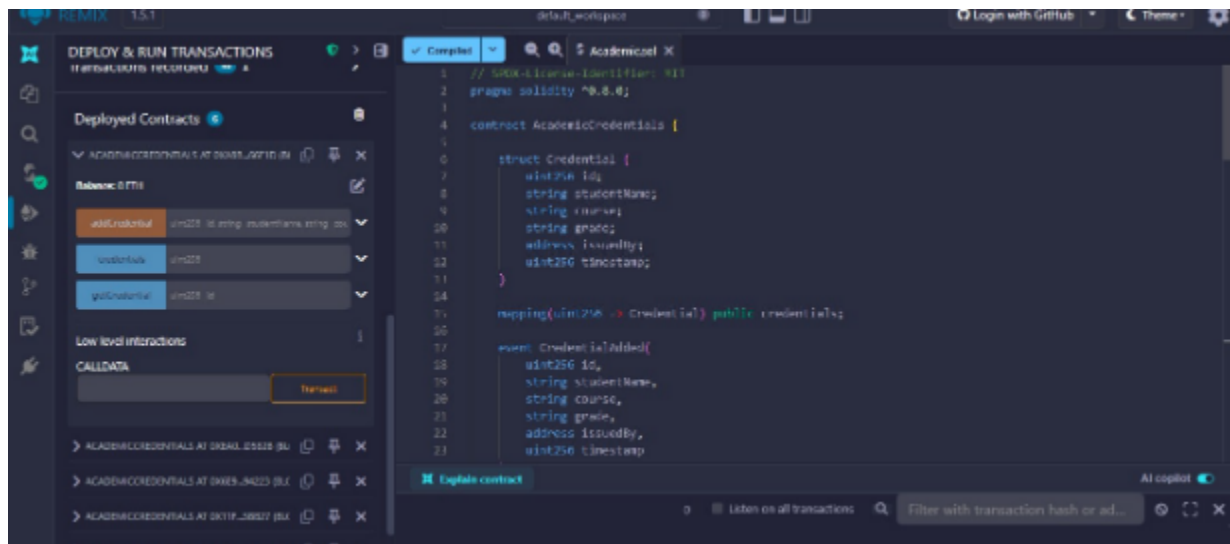
function getCredential(uint256 _id) public view returns ( uint256, string memory, string memory,
string memory, address, uint256)
{
    Credential memory c = credentials[_id]; return (c.id, c.studentName, c.course, c.grade,
c.issuedBy, c.timestamp);
}
}
```

Screenshots :-

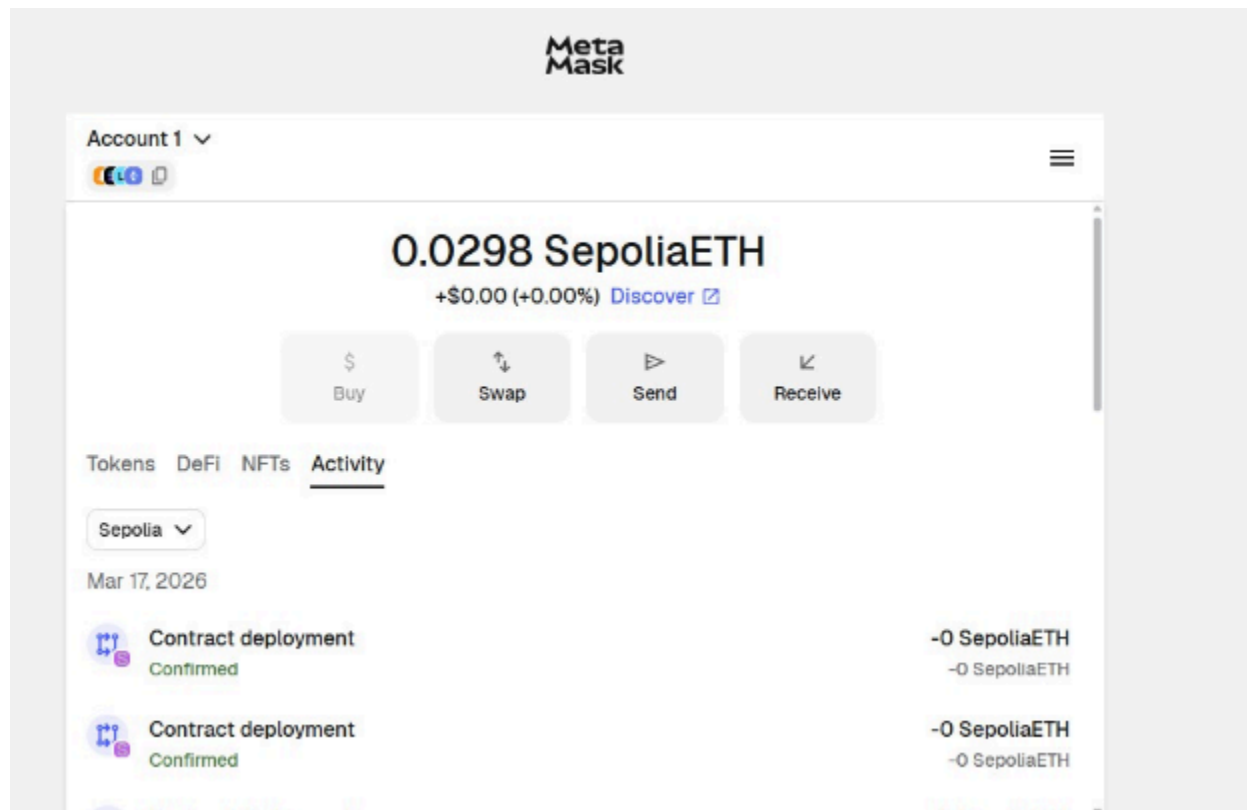
Metamask connected :



Contract deployed :



Transaction completed :



Conclusion :-

The experiment successfully demonstrated the integration of MetaMask with Remix IDE and execution of blockchain transactions. The smart contract was deployed on Sepolia Testnet and used to store and verify academic credentials securely. This shows how blockchain can be used for tamper-proof academic verification systems.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Exp 8

Aim :- To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory :-

1. Understanding Ganache as a Local Blockchain Platform

Ganache is a personal Ethereum blockchain used for development and testing. It simulates a real blockchain environment but runs locally on your computer. This allows developers to deploy contracts, test transactions, and debug without spending real cryptocurrency.

Key characteristics of Ganache :

- It provides a set of pre-funded test accounts.
- It runs a local Ethereum node instantly without syncing.
- It allows inspection of blocks, transactions, gas usage, and contract events.
- It is ideal for rapid prototyping and debugging before deploying to public testnets or mainnet.

Ganache is part of the Truffle Suite and is widely used in Ethereum development workflows. It acts as a controlled environment where developers can repeatedly deploy and test smart contracts without cost.

2. Role of MetaMask in Smart Contract Deployment

MetaMask is a browser-based Ethereum wallet and gateway to blockchain networks. It manages private keys, signs transactions, and connects decentralized applications to networks such as :

- Local blockchain (Ganache)
- Public testnets (Goerli, Sepolia)
- Ethereum mainnet

When using Ganache, MetaMask is configured to connect to the local RPC server (usually <http://127.0.0.1:7545> ([127.0.0.1 in Bing](#))). The accounts generated by Ganache can be imported into MetaMask using their private keys. This allows MetaMask to sign and send transactions to the local blockchain.

MetaMask acts as the bridge between Remix IDE and Ganache by authorizing deployments and contract interactions.

3. Remix IDE as a Smart Contract Development Environment

Remix IDE is a browser-based platform for writing, compiling, deploying, and testing Solidity smart contracts. It provides :

- A Solidity editor with syntax highlighting
- A compiler for generating bytecode and ABI
- Deployment tools that connect to MetaMask
- A user interface for interacting with deployed contracts
- Debugging and analysis tools

Remix can deploy contracts to :

- JavaScript VM (in-browser blockchain)
- Injected Provider (MetaMask)
- External networks (Ganache, testnets, mainnet)

When MetaMask is connected to Ganache, Remix uses MetaMask as the transaction signer and Ganache as the execution environment.

4. Deploying a Smart Contract Using Ganache, MetaMask, and Remix

The deployment process involves several conceptual steps :

1. Writing the smart contract
A Solidity contract is created defining state variables, functions, and logic.
2. Compiling the contract
Remix compiles the Solidity code into EVM bytecode and generates the ABI (Application Binary Interface).
The ABI defines how external applications interact with the contract.
3. Connecting Remix to MetaMask
Remix uses the Injected Provider option to route deployment transactions through MetaMask.
4. MetaMask connecting to Ganache
MetaMask is configured to use Ganache's RPC endpoint.
When Remix sends a deployment request, MetaMask signs the transaction using the selected account.

5. Deployment to Ganache

Ganache receives the signed transaction, mines a block instantly, and stores the contract on the local blockchain. A contract address is generated.

6. Interacting with the contract

Remix displays the deployed contract's functions.

Each function call becomes a transaction (if it modifies state) or a call (if it is read-only).

Ganache logs all transactions, gas usage, and events for inspection.

This workflow allows developers to test contract logic thoroughly before deploying to real networks.

5. Example of a Simple Solidity Contract

A minimal example of a contract that can be deployed using this workflow :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {
    uint public value;

    function setValue(uint _value) public {
        value = _value;
    }
}
```

This contract demonstrates :

- A state variable stored on the blockchain
 - A function that modifies state (requires a transaction)
 - A public getter function automatically generated by Solidity
-

6. Summary of the Complete Workflow

1. Install and run Ganache to create a local blockchain.
2. Open MetaMask and configure it to connect to Ganache's RPC network.
3. Import Ganache accounts into MetaMask using private keys.
4. Write and compile the Solidity contract in Remix IDE.
5. Select Injected Provider (MetaMask) in Remix.
6. Deploy the contract; MetaMask signs the transaction.
7. Ganache mines the transaction and deploys the contract.

- Interact with the contract through Remix while Ganache logs all activity.

Code :-

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.0;

contract CharityDonation {
    address public owner; uint public totalDonations;
    struct Donation {
        address donor; uint amount; uint timestamp;
    }

    Donation[] public donations;

    event DonationReceived(address donor, uint amount); event FundsWithdrawn(address owner,
    uint amount);

    constructor() {
        owner = msg.sender;
    }

    // Function to donate Ether function donate() public payable {
        require(msg.value > 0, "Donation must be greater than zero");

        donations.push(Donation(msg.sender, msg.value, block.timestamp)); totalDonations +=
        msg.value;

        emit DonationReceived(msg.sender, msg.value);
    }

    // Function to check contract balance
    function getBalance() public view returns (uint) {
        return address(this).balance;
    }

    // Only owner can withdraw funds function withdraw(uint amount) public {
        require(msg.sender == owner, "Only owner can withdraw funds"); require(amount <=
        address(this).balance, "Insufficient balance");

        payable(owner).transfer(amount);
        emit FundsWithdrawn(owner, amount);
    }
}
```

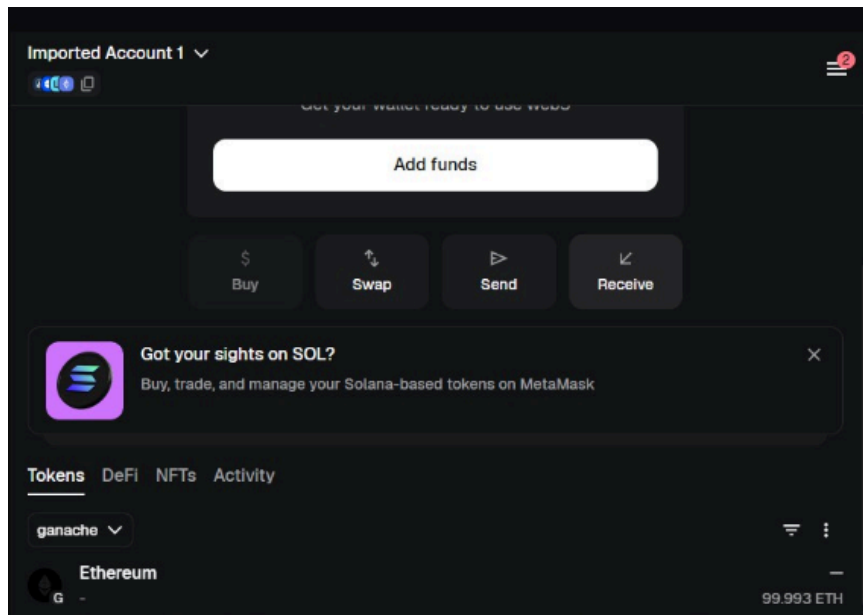
// View number of donations

```
function getDonationCount() public view returns (uint) {  
    return donations.length;  
}
```

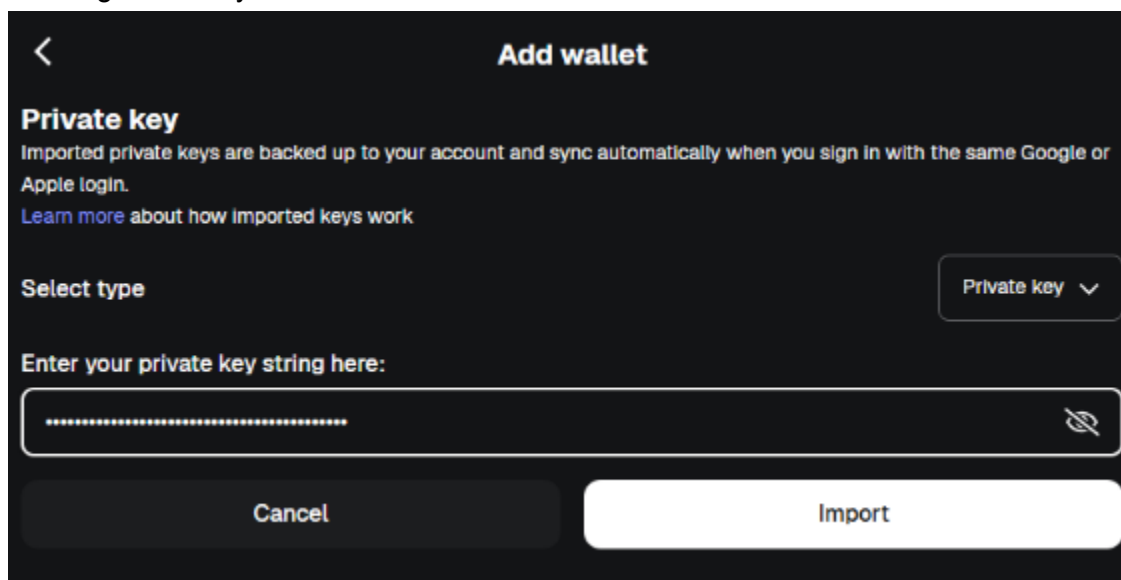
}
Metamask

Screenshots :-

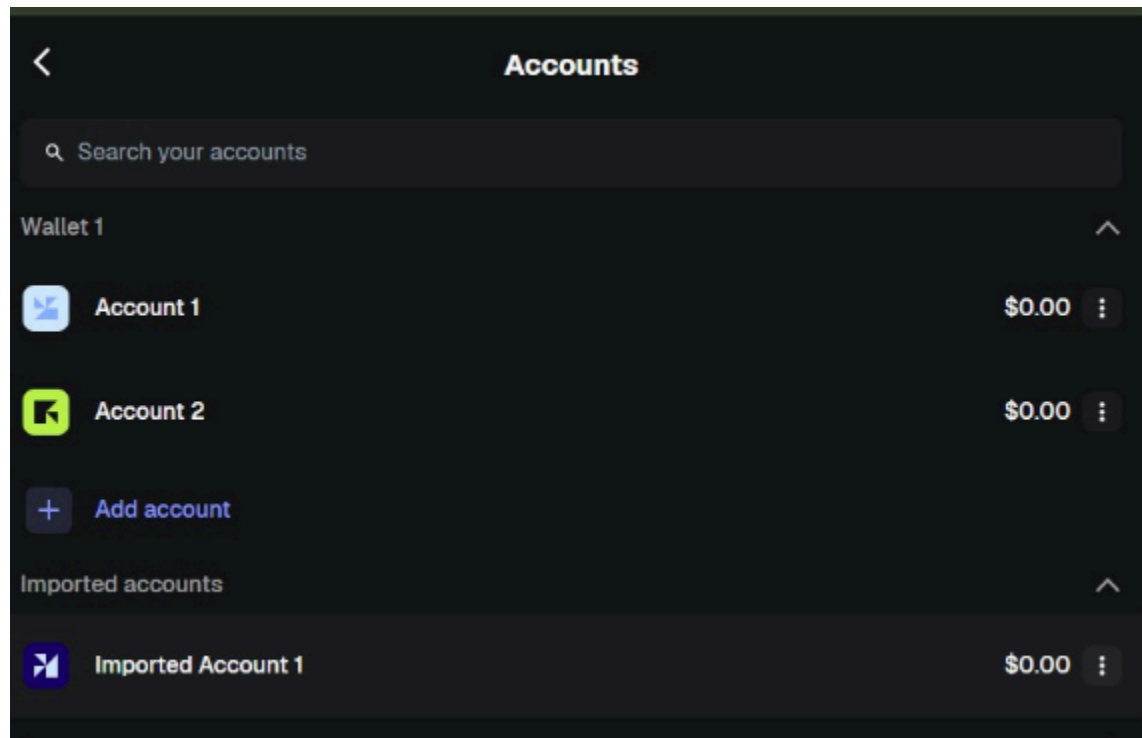
Adding funds for testing :



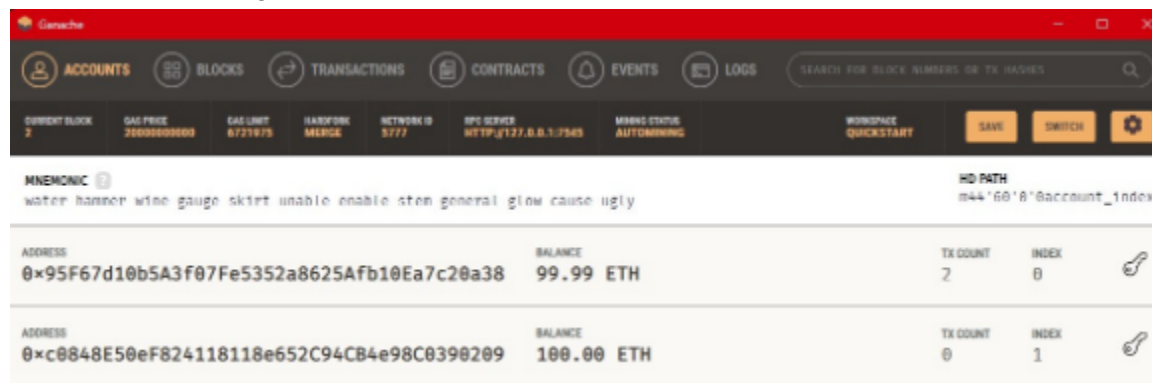
Entering wallet keys :



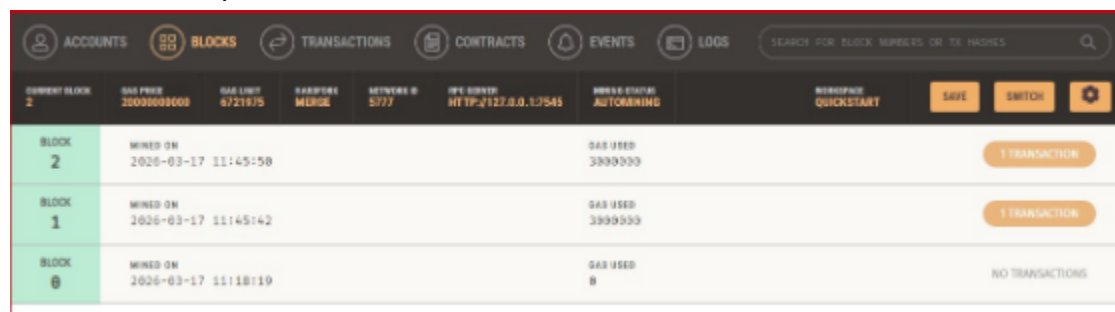
Accounts for the funds to be added to for testing :



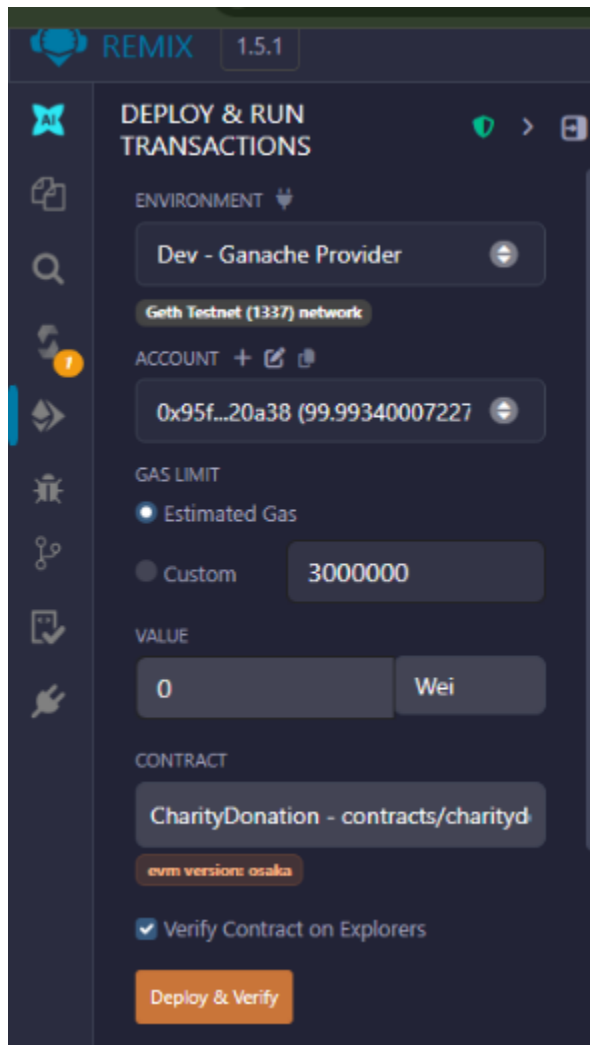
Transaction pending :



Transaction completed :



Contract deployed and executed :



Conclusion :-

This experiment demonstrated the implementation of a blockchain-based environment using Ganache, MetaMask, and Remix IDE. A smart contract was successfully created, deployed, and tested using Ganache accounts integrated with MetaMask. The experiment also highlighted the potential of blockchain technology in enhancing charity donation systems by ensuring transparency, security, and traceability of transactions. Overall, it provided practical knowledge of executing and monitoring blockchain transactions in a controlled environment.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Exp 9

Aim - To implement a Private Ethereum Blockchain using Geth.

Theory -

Ethereum is an open-source, decentralized blockchain platform that enables developers to build and deploy smart contracts and decentralized applications (DApps). Unlike public blockchains such as the Ethereum mainnet, a Private Ethereum Blockchain is restricted to specific participants, making it ideal for development, testing, and enterprise applications where confidentiality, control, and performance are essential.

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through JSON-RPC, command-line, or JavaScript consoles.

Setting up a Private Ethereum Network involves creating a customized blockchain environment isolated from the public Ethereum network. It enables developers to test smart contracts, consensus mechanisms, and network behavior without incurring gas costs or depending on public validators.

Steps Involved:

1. **Choosing a Network ID:**
2. Each private blockchain must have a unique network ID to differentiate it from public or other private networks.
3. **Choosing a Consensus Algorithm:**
4. Ethereum supports different consensus mechanisms such as **Proof of Work (PoW)** and **Proof of Authority (PoA)**. In private networks, **Clique (PoA)** is often used for faster block confirmation.
5. **Creating a Genesis Block:**
6. The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.
7. **Initializing the Geth Database:**
8. The `geth init` command initializes the blockchain using the **genesis.json** file, preparing the data directory for node operation.
9. **Setting up Networking:**
10. Nodes communicate using **enodes** (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.
11. **Running the Member Nodes:**

12. Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.
13. **Running a Signer (in Clique):**
14. In the **Clique consensus**, designated **signers** validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

Outputs -

1. First install all the necessary dependencies like curl, git, build-essential.
`sudo apt install wget curl git build-essential -y`

```
deilsy@deilsy:~$ sudo apt install wget curl git build-essential -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.4-1ubuntu4.1).
wget set to manually installed.
git is already the newest version (1:2.43.0-1ubuntu7.3).
build-essential is already the newest version (12.10ubuntu1).
The following additional packages will be installed:
  libcurl3t64-gnutls libcurl4t64
The following packages will be upgraded:
  curl libcurl3t64-gnutls libcurl4t64
3 upgraded, 0 newly installed, 0 to remove and 57 not upgraded.
Need to get 902 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 curl amd64 8.5.0-2ubuntu10.8 [227 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libcurl4t64 amd64 8.5.0-2ubuntu10.8 [342 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libcurl3t64-gnutls amd64 8.5.0-2ubuntu10.8 [334 kB]
Fetched 902 kB in 1s (650 kB/s)
(Reading database ... 46694 files and directories currently installed.)
Preparing to unpack .../curl_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking curl (8.5.0-2ubuntu10.8) over (8.5.0-2ubuntu10.6) ...
Preparing to unpack .../libcurl4t64_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking libcurl4t64:amd64 (8.5.0-2ubuntu10.8) over (8.5.0-2ubuntu10.6) ...
Preparing to unpack .../libcurl3t64-gnutls_8.5.0-2ubuntu10.8_amd64.deb ...
Unpacking libcurl3t64-gnutls:amd64 (8.5.0-2ubuntu10.8) over (8.5.0-2ubuntu10.6) ...
Setting up libcurl4t64:amd64 (8.5.0-2ubuntu10.8) ...
Setting up libcurl3t64-gnutls:amd64 (8.5.0-2ubuntu10.8) ...
Setting up curl (8.5.0-2ubuntu10.8) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
```

2. Install geth using wget `cd /usr/local/bin && sudo wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz`

```

deilsy@delsy:~$ cd /usr/local/bin && sudo wget https://gethstore.blob.core.w
indows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz
--2026-03-26 03:54:03-- https://gethstore.blob.core.windows.net/builds/geth
-linux-amd64-1.12.0-e501b3b0.tar.gz
Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net).
.. 20.60.40.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.n
et)|20.60.40.164|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 26856444 (26M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz'

geth-linux-amd64-1 100%[=====>] 25.61M 734KB/s in 27s

2026-03-26 03:54:32 (968 KB/s) - 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz' s
aved [26856444/26856444]

deilsy@delsy:/usr/local/bin$ ls
geth-linux-amd64-1.12.0-e501b3b0.tar.gz
deilsy@delsy:/usr/local/bin$

```

3. Extract the tar file and move the geth into /usr/local/bin/geth path & check the geth version

```

sudo tar -xvf geth-linux-amd64-1.12.0-e501b3b0.tar.gz sudo mv
geth-linux-amd64-1.12.0-e501b3b0/geth /usr/local/bin/geth geth version

```

```

deilsy@delsy:/usr/local/bin$ sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /
usr/local/bin/geth
deilsy@delsy:/usr/local/bin$ geth version
Geth
Version: 1.12.0-stable
Git Commit: e501b3b05db8e169f67dc78b7b59bc352b3c638d
Git Commit Date: 20230525
Architecture: amd64
Go Version: go1.20.3
Operating System: linux
GOPATH=
GOROOT=
deilsy@delsy:/usr/local/bin$

```

4. Create a folder named private_ethereum_setup and create another 2 directories naming node1 and node2 individually inside private_ethereum_setup directory.

```

mkdir private_ethereum_setup
cd private_ethereum_setup && mkdir node1 node2

```

```

deilsy@delsy:~$ ls private_ethereum_setup/
node1 node2
deilsy@delsy:~$

```

5. Check the files
6. Create account for node1
geth --datadir "node1" account new

```

deilsy@deilsy:~/private_ethereum_setup$ geth --datadir "node1" account new
INFO [03-26|04:01:54.815] Maximum peer count          ETH=50 LE
S=0 total=50
INFO [03-26|04:01:54.816] Smartcard socket not found, disabling   err="stat
/run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not f
orget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x563A5a670260417B163AD1696fCBfA6849906d06
Path of the secret key file: node1/keystore/UTC--2026-03-26T04-02-01.7653580
44Z--563a5a670260417b163ad1696fcbfa6849906d06

- You can share your public address with anyone. Others need it to interact
with you.
- You must NEVER share the secret key with anyone! The key controls access t
o your funds!
- You must BACKUP your key file! Without the key, it's impossible to access
account funds!
- You must REMEMBER your password! Without the password, it's impossible to
decrypt the key!

deilsy@deilsy:~/private_ethereum_setup$

```

7. Do the same for node2

geth --datadir "node2" account new

```

deilsy@deilsy:~/private_ethereum_setup$ geth --datadir "node2" account new
INFO [03-26|04:02:43.813] Maximum peer count          ETH=50 LE
S=0 total=50
INFO [03-26|04:02:43.813] Smartcard socket not found, disabling   err="stat
/run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not f
orget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x6d04eA66725A40F5669E2d6f4f77eb4EcF756a3f
Path of the secret key file: node2/keystore/UTC--2026-03-26T04-02-49.2597870
89Z--6d04ea66725a40f5669e2d6f4f77eb4ecf756a3f

- You can share your public address with anyone. Others need it to interact
with you.
- You must NEVER share the secret key with anyone! The key controls access t
o your funds!
- You must BACKUP your key file! Without the key, it's impossible to access
account funds!
- You must REMEMBER your password! Without the password, it's impossible to
decrypt the key!

```

8. Create a file called genesis.json in the same directory and copy paste these contents

```

{
  "config": { "chainId": 12345,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,

```

```

    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "istanbulBlock": 0,
    "muirGlacierBlock": 0,
    "berlinBlock": 0,
    "londonBlock": 0,
    "arrowGlacierBlock": 0,
    "grayGlacierBlock": 0, "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "800000000", "extradata":
  "0x0000000000000000000000000000000000000000000000000000000000000000
  0000000098608ADf9c785d54f40cDcf6700E990771b1922600000000000000
  0000000000000000000000000000000000000000000000000000000000000000
  0000000000000000000000000000000000000000000000000000000000000000
  0",
  "alloc": { "0x98608ADf9c785d54f40cDcf6700E990771b19226": {
    "balance": "1000000000000000000000000" },
    "0x7B25e791D24A3F5c453A9E5468cF6cEa2243092C": {
      "balance": "1200000000000000000000000" }
  }
}

```

Make changes in the alloc part replace the public keys of the account that you have generated in earlier steps.

```

{
  "config": {
    "chainId": 12345,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "istanbulBlock": 0,
    "muirGlacierBlock": 0,
    "berlinBlock": 0,
    "londonBlock": 0,
    "arrowGlacierBlock": 0,
    "grayGlacierBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000000",
  "extradata": "0x0000000000000000000000000000000000000000000000000000000000000000",
  "alloc": {
    "0x563A5a670260417B163AD1696FCBfA6849906d06": { "balance": "10000000000",
    "0x6d04eA66725A40F5669E2d6F4f77eb4Ec756a3f": { "balance": "12000000000"
  }
}

```

9. Initializing geth for node1 using the genesis.json file
 geth init --datadir node1 genesis.json

```

deilsy@deilsy: ~/private_ethereum_setup$ geth init --datadir node1 genesis.js
n
INFO [03-26|04:06:53.763] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:06:53.764] Smartcard socket not found, disabling   err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:06:53.767] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:06:53.768] Initializing the KZG library            backend=g
okzg
INFO [03-26|04:06:53.810] Defaulting to pebble as the backing database
INFO [03-26|04:06:53.810] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata cache=16.00MiB hand
les=16
INFO [03-26|04:06:53.957] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:06:53.957] Writing custom genesis block
INFO [03-26|04:06:53.958] Persisted trie from memory database    nodes=3 s
ize=413.00B time="100.853µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 li
vesize=0.00B
INFO [03-26|04:06:53.976] Successfully wrote genesis state        database=
chaindata hash=732ee4..dabc1f
INFO [03-26|04:06:53.976] Defaulting to pebble as the backing database
INFO [03-26|04:06:53.976] Allocated cache and file handles       database=
/home/deilsy/private_ethereum_setup/node1/geth/lightchaindata cache=16.00MiB
handles=16
INFO [03-26|04:06:54.074] Opened ancient database                 database=
/home/deilsy/private_ethereum_setup/node1/geth/lightchaindata/ancient/chain
readonly=false
INFO [03-26|04:06:54.074] Writing custom genesis block
INFO [03-26|04:06:54.074] Persisted trie from memory database    nodes=3 s
ize=413.00B time="18.276µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 li
vesize=0.00B
INFO [03-26|04:06:54.086] Successfully wrote genesis state        database=
lightchaindata hash=732ee4..dabc1f
deilsy@deilsy: ~/private_ethereum_setup$ █

```

10. Do the same for node2

geth init --datadir node2 genesis.json

```

deilsy@deilsy:~/private_ethereum_setup$ geth init --datadir node2 genesis.js
n
INFO [03-26|04:07:16.531] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:07:16.531] Smartcard socket not found, disabling  err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:07:16.534] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:07:16.534] Initializing the KZG library           backend=g
okzg
INFO [03-26|04:07:16.580] Defaulting to pebble as the backing database
INFO [03-26|04:07:16.580] Allocated cache and file handles      database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata cache=16.00MiB hand
les=16
INFO [03-26|04:07:16.713] Opened ancient database                database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:07:16.713] Writing custom genesis block
INFO [03-26|04:07:16.713] Persisted trie from memory database    nodes=3 s
ize=413.00B time="23.604µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 liv
esize=0.00B
INFO [03-26|04:07:16.724] Successfully wrote genesis state        database=
chaindata hash=732ee4..dabc1f
INFO [03-26|04:07:16.724] Defaulting to pebble as the backing database
INFO [03-26|04:07:16.724] Allocated cache and file handles      database=
/home/deilsy/private_ethereum_setup/node2/geth/lightchaindata cache=16.00MiB
handles=16
INFO [03-26|04:07:16.799] Opened ancient database                database=
/home/deilsy/private_ethereum_setup/node2/geth/lightchaindata/ancient/chain
readonly=false
INFO [03-26|04:07:16.800] Writing custom genesis block
INFO [03-26|04:07:16.800] Persisted trie from memory database    nodes=3 s
ize=413.00B time="29.651µs" gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 liv
esize=0.00B
INFO [03-26|04:07:16.814] Successfully wrote genesis state        database=
lightchaindata hash=732ee4..dabc1f

```

11. Now we will install bootnode for configuring both the nodes.

```

sudo add-apt-repository -y ppa:ethereum/ethereum
sudo apt-get update
sudo apt-get install bootnode

```

12. Create a key for the bootnode and save it to boot.key in the folder bootnode

```

-genkey boot.key

```

```

deilsy@deilsy:~/private_ethereum_setup$ bootnode -genkey boot.key
deilsy@deilsy:~/private_ethereum_setup$ ls
boot.key  genesis.json  node1  node2
deilsy@deilsy:~/private_ethereum_setup$

```

13. Copy this command and copy the enode://.... This will be required later

```

bootnode -nodekey boot.key -verbosity 9 -addr :30305

```

```
deilsy@deilsy:~/private_ethereum_setup$ bootnode -nodekey boot.key -verbosity
9 -addr :30305
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f51333118e81f54e7
5919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.0.1:0?dis
cport=30305
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deploymen
ts.
INFO [03-26|04:20:52.969] New local node record                      seq=1,774
,498,852,968 id=7212962da899bfc0 ip="invalid IP" udp=0 tcp=0
```

14. Now on another terminal we will start mining node1

```
geth --datadir node1 --port 30306 --bootnodes
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f5133
3118e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc7
7ab41f27@127.0.0.1:0?discport=30305 --networkid 123454321 -- unlock
0x563A5a670260417B163AD1696fCBfA6849906d06 --password node1/password.txt
--authrpc.port 8551 --miner.etherbase
0x563A5a670260417B163AD1696fCBfA6849906d06 --mine
```

Replace the address with your own created

```

deilsy@delsy:~/private_ethereum_setup$ geth --datadir node1 --port 30306 --b
ootnodes enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f5133311
8e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.
0.1:0?discport=30305 --networkid 123454321 --unlock 0x563A5a670260417B163AD1
696fCBfA6849906d06 --password node1/password.txt --authrpc.port 8551 --miner
.etherbase 0x563A5a670260417B163AD1696fCBfA6849906d06 --mine
INFO [03-26|04:28:57.471] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:28:57.472] Smartcard socket not found, disabling      err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:28:57.474] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:28:57.474] Initializing the KZG library                backend=g
okzg
INFO [03-26|04:28:57.529] Allocated trie memory caches              clean=154
.00MiB dirty=256.00MiB
INFO [03-26|04:28:57.529] Using pebble as the backing database
INFO [03-26|04:28:57.530] Allocated cache and file handles        database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata cache=512.00MiB han
dles=524,288
INFO [03-26|04:28:57.629] Opened ancient database                  database=
/home/deilsy/private_ethereum_setup/node1/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:28:57.641] Initialising Ethereum protocol          network=1
23,454,321 dbversion=8
INFO [03-26|04:28:57.641] -----
-----
INFO [03-26|04:28:57.641] Chain ID: 12345 (unknown)
INFO [03-26|04:28:57.641] Consensus: Clique (proof-of-authority)

```

15. Now do same for node2

```

geth --datadir node2 --port 30307 --bootnodes
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917
f51333118e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af55
4ccb922bc77ab41f27@127.0.0.1:0?discport=30305 --networkid 123454321 --unlock
0x6d04eA66725A40F5669E2d6f4f77eb4Ecf756a3f
--password node2/password.txt --authrpc.port 8552

```

```

deilsy@deilsy:~/private_ethereum_setup$ geth --datadir node2 --port 30307 --b
ootnodes enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f5133311
8e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.
0.1:0?discport=30305 --networkid 123454321 --unlock 0x6d04eA66725A40F5669E2d
6f4f77eb4Ecf756a3f --password node2/password.txt --authrpc.port 8552
INFO [03-26|04:35:17.744] Maximum peer count                      ETH=50 LE
S=0 total=50
INFO [03-26|04:35:17.745] Smartcard socket not found, disabling   err="stat
/run/pcscd/pcscd.comm: no such file or directory"
INFO [03-26|04:35:17.748] Set global gas cap                      cap=50,00
0,000
INFO [03-26|04:35:17.748] Initializing the KZG library            backend=g
okzg
INFO [03-26|04:35:17.795] Allocated trie memory caches          clean=154
.00MiB dirty=256.00MiB
INFO [03-26|04:35:17.795] Using pebble as the backing database
INFO [03-26|04:35:17.795] Allocated cache and file handles      database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata cache=512.00MiB han
dles=524,288
INFO [03-26|04:35:17.928] Opened ancient database                database=
/home/deilsy/private_ethereum_setup/node2/geth/chaindata/ancient/chain reado
nly=false
INFO [03-26|04:35:17.929] Initialising Ethereum protocol        network=1
23,454,321 dbversion=<nil>
INFO [03-26|04:35:17.930]
INFO [03-26|04:35:17.930] -----
-----
INFO [03-26|04:35:17.930] Chain ID: 12345 (unknown)
INFO [03-26|04:35:17.930] Consensus: Clique (proof-of-authority)
INFO [03-26|04:35:17.930]
INFO [03-26|04:35:17.930] Pre-Merge hard forks (block based):
INFO [03-26|04:35:17.930]   - Homestead: #0 (https:/
/github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-up
grades/homestead.md)

```

16. Now we will interact with the blockchain using javascript console geth attach node1/geth.ipc

```

deilsy@deilsy:~/private_ethereum_setup$ geth attach node1/geth.ipc
Welcome to the Geth JavaScript console!

instance: Geth/v1.12.0-stable-e501b3b0/linux-amd64/go1.20.3
coinbase: 0x563a5a670260417b163ad1696fcbfa6849906d06
at block: 105 (Thu Mar 26 2026 04:37:38 GMT+0000 (UTC))
  datadir: /home/deilsy/private_ethereum_setup/node1
  modules: admin:1.0 clique:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.
0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> █

```

17. Interacting with the blockchain

```
> net.peerCount
1
> eth.accounts
["0x563a5a670260417b163ad1696fcbfa6849906d06"]
> eth.accounts[0]
"0x563a5a670260417b163ad1696fcbfa6849906d06"
> eth.blockNumber
302
> eth.getBalance(eth.accounts[0])
1e+23
> web3.fromWei(eth.getBalance("0x563A5a670260417B163AD1696fCBfA6849906d06"),
"ether")
100000
```



```

> web3.fromWei(eth.getBalance("0x6d04eA66725A40F5669E2d6f4f77eb4Ec756a3f"),
"ether")
120010
> web3.eth.getTransaction("0x3cb6cc9cd65c927b0a0af8351e757ef8fdf76ea00d66b88
63b3c9fd9c3657508")
{
  accessList: [],
  blockHash: "0xe772e9613d2f39454f41076dd9e740948bac2e0ecfa254702b4f474fbecc
a438",
  blockNumber: 338,
  chainId: "0x3039",
  from: "0x563a5a670260417b163ad1696fcbfa6849906d06",
  gas: 30000,
  gasPrice: 1000000007,
  hash: "0x3cb6cc9cd65c927b0a0af8351e757ef8fdf76ea00d66b8863b3c9fd9c3657508"
,
  input: "0x",
  maxFeePerGas: 10000000014,
  maxPriorityFeePerGas: 10000000000,
  nonce: 0,
  r: "0x84be672e734bdae2f6d262abda2a08687028a86e035d8c73c8c80662fd6c64fe",
  s: "0x196df54d8f571bde60289c94926793a29f05947203c25418295a1cd80c7f9c6a",
  to: "0x6d04ea66725a40f5669e2d6f4f77eb4ecf756a3f",
  transactionIndex: 0,
  type: "0x2",
  v: "0x0",
  value: 10000000000000000000
}
>

```

Conclusion - Private blockchain was created using geth, the nodes in the blockchain were configured and we interacted with the blockchain by making a transaction of 10 ethers.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Exp 10

Aim: Explore the Cryptocurrency Landscape

Theory:

Cryptocurrency Landscape – Theory

:

Cryptocurrencies are decentralized digital assets that operate on blockchain technology, enabling secure, transparent, and immutable transactions without the need for a central authority. These systems rely on distributed ledgers where transactions are validated through consensus mechanisms and recorded across a network of nodes.

Data Sources

Reliable and diverse data sources are essential for understanding cryptocurrency markets:

- Financial Data Platforms – Services like CoinGecko and CoinMarketCap provide structured data such as market capitalization, circulating supply, trading volume, and historical price trends.
- News Aggregators – Platforms such as CryptoPanic collect real-time news from multiple sources, helping track major events impacting the market.
- Social Sentiment Analytics – Tools like LunarCrush analyze social media activity to measure public sentiment and engagement.
- Regulatory Sources – Official portals like the U.S. Securities and Exchange Commission and frameworks such as MiCA provide insights into legal policies and compliance requirements.

Types of Cryptocurrencies

The cryptocurrency ecosystem consists of various categories based on functionality: • Store of Value (SoV) – Bitcoin (BTC), known for limited supply and long-term value preservation.

- Utility Tokens – Ether (ETH) and Chainlink (LINK), used for network operations and smart contract execution.
- Stablecoins – Tether (USDT) and USD Coin (USDC), designed to maintain price stability by being pegged to fiat currencies.
- Governance Tokens – Uniswap (UNI) and Aave (AAVE), enabling users to participate in decentralized decision-making.

Key Characteristics

Cryptocurrencies exhibit the following features:

- Decentralization – No central authority controls the system.
- Transparency – Transactions are publicly recorded on the blockchain.
- Immutability – Data cannot be altered once confirmed.
- Security – Cryptographic algorithms ensure data integrity and safety.

Market Dynamics

The cryptocurrency market is influenced by multiple factors:

- Volatility – Prices fluctuate significantly due to demand, sentiment, and external events.
- Liquidity – Depends on trading volume and market participation.
- Market Sentiment – News, social media, and investor behavior impact price trends.
- Regulation – Government policies and frameworks influence adoption and stability.

Blockchain Technology Advancements

Modern blockchain systems are evolving to overcome limitations:

- Consensus Mechanisms – Shift from Proof of Work (PoW) to Proof of Stake (PoS) for efficiency.
- Scalability Solutions – Layer-2 technologies and sharding improve transaction speed.
- Interoperability – Enables communication between different blockchain networks.
- Privacy Enhancements – Zero-Knowledge Proofs (ZKP) allow secure verification without revealing sensitive data.

Adoption and Emerging Trends

Cryptocurrency adoption continues to grow across sectors:

- Digital Payments – Companies like Stripe and Shopify support crypto transactions.
- Decentralized Finance (DeFi) – Financial services operate without intermediaries.
- Non-Custodial Wallets – MetaMask and Phantom give users full asset control.
- Asset Tokenization – Real-world assets are increasingly represented on blockchain networks.

Utilizing Api for analysis and visualization :

Code:

```
!pip install requests pandas matplotlib
```

```
# Imports import
requests import pandas
as pd import
matplotlib.pyplot as plt

# ----- CONFIG-----
API_KEY = "api-key"
COIN = "BTC"
CURRENCY = "USD"
DAYS = 180

# ----- FETCH FUNCTION      def
fetch_data(coin, currency, days):
    url = "https://min-api.cryptocompare.com/data/v2/histoday"

    params = {
        "fsym":
            coin,
        "tsym": currency,
        "limit": days,
        "api_key":
            API_KEY
    }

    res = requests.get(url,
params=params) data = res.json()

    if data["Response"] != "Success":
print("Error:", data.get("Message"))
return None

    return data["Data"]["Data"]

# ----- LOAD DATA      raw =
fetch_data(COIN, CURRENCY, DAYS) df =
pd.DataFrame(raw)
```

```

# Time formatting
df["date"] = pd.to_datetime(df["time"], unit="s")
df.set_index("date", inplace=True)

# Core columns df["price"]
= df["close"] df["volume"] =
df["volumeto"]

# ----- CALCULATIONS      df["returns"] =
df["price"].pct_change() df["MA_7"] =
df["price"].rolling(7).mean() df["MA_21"] =
df["price"].rolling(21).mean()
df["cumulative_return"]
= (1 + df["returns"]).cumprod() df["volatility"] =
df["returns"].rolling(7).std() * 100

# ----- PLOTTING (6 GRAPHS)      plt.figure(figsize=(16,
12))

# 1. Price Trend
plt.subplot(3, 2, 1)
plt.plot(df["price"])
plt.title("1. Price Trend")
plt.grid()

# 2. Price vs Volume plt.subplot(3, 2, 2)
plt.plot(df["price"], label="Price") plt.bar(df.index,
df["volume"], alpha=0.3, label="Volume")
plt.legend() plt.title("2. Price vs Volume")

# 3. Returns Distribution
plt.subplot(3, 2, 3)
df["returns"].dropna().hist(bins=40)
plt.title("3. Returns Distribution")

# 4. Moving Averages
plt.subplot(3, 2, 4)

```

```
plt.plot(df["price"], label="Price", alpha=0.5)
plt.plot(df["MA_7"], label="7-Day MA")
plt.plot(df["MA_21"], label="21-Day MA") plt.legend()
plt.title("4. Moving Averages")
```

```
# 5. Cumulative Growth
plt.subplot(3, 2, 5)
plt.plot(df["cumulative_return"])
plt.title("5. Cumulative Returns")
```

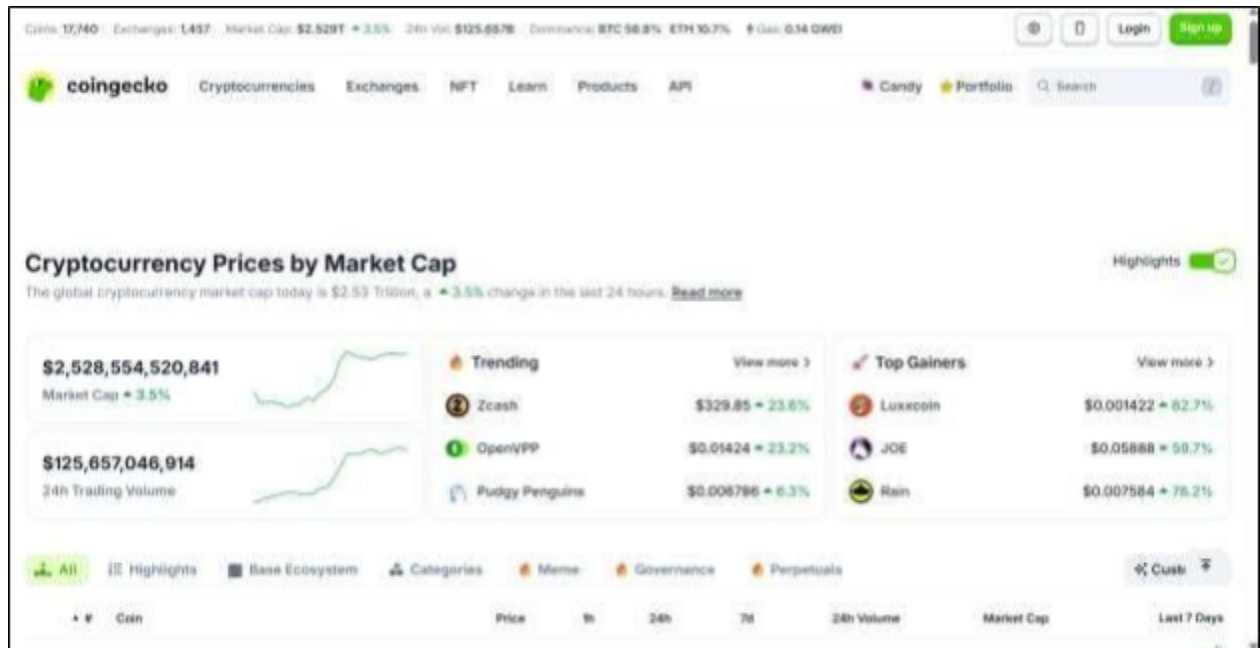
```
# 6. Volatility
plt.subplot(3, 2, 6)
plt.plot(df["volatility"])
plt.title("6. Volatility
(%)")
```

```
plt.tight_layout() plt.show()
```

Output:-

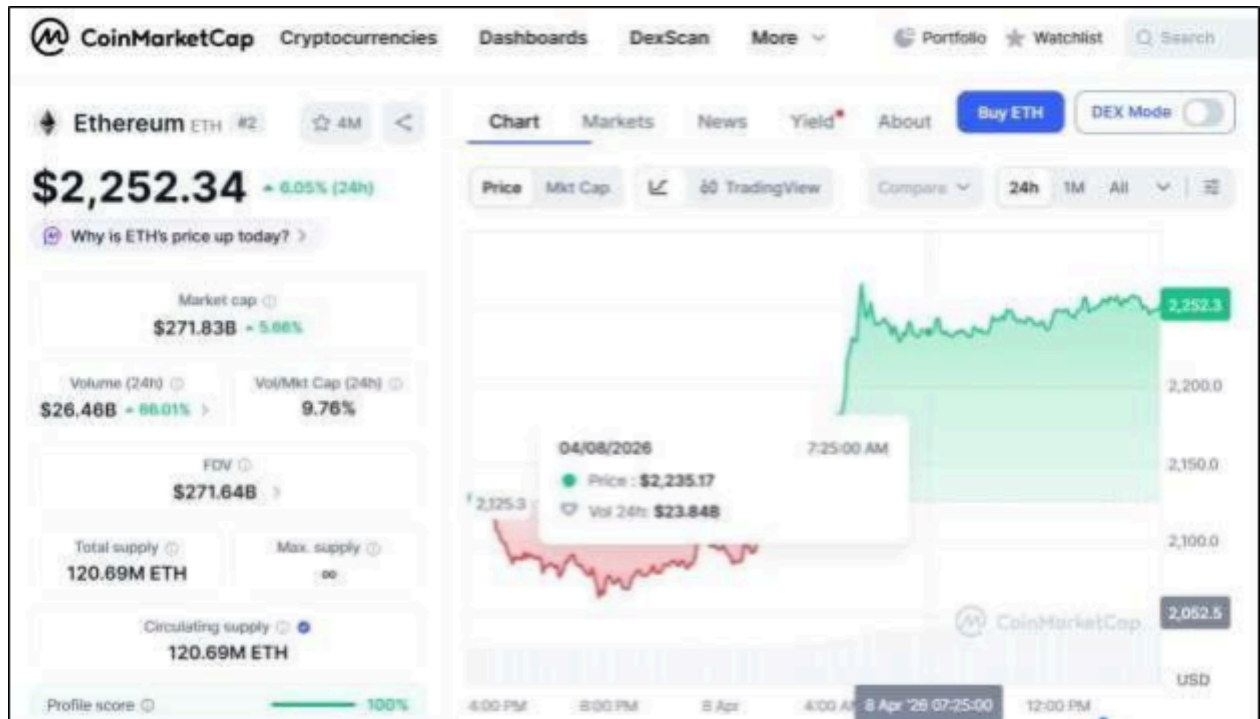


#	Coin	Price	1h	24h	7d	24h Volume	Market Cap	Last 7 Days
1	Bitcoin BTC	\$71,650.57	+0.3%	+3.8%	+4.4%	\$53,774,493,979	\$1,435,255,753,725	
2	Ethereum ETH	\$2,249.04	+0.5%	+5.9%	+5.2%	\$26,616,741,403	\$271,670,437,052	
3	Tether USDT	\$0.9999	+0.0%	+0.0%	+0.0%	\$86,609,739,782	\$184,136,015,689	
4	XRP XRP	\$1.38	+0.1%	+4.7%	+2.3%	\$3,756,388,262	\$54,908,252,063	
5	BNB BNB	\$612.05	+0.3%	+1.3%	+0.5%	\$1,393,859,290	\$83,489,901,411	
6	USDC USDC	\$0.9992	+0.1%	+0.1%	+0.1%	\$16,120,644,245	\$77,902,364,173	
7	Solana SOL	\$84.61	+0.2%	+5.3%	+1.5%	\$5,168,686,009	\$48,577,651,738	
8	TRON TRX	\$0.3164	+0.0%	+0.4%	+0.4%	\$478,869,461	\$29,988,114,949	

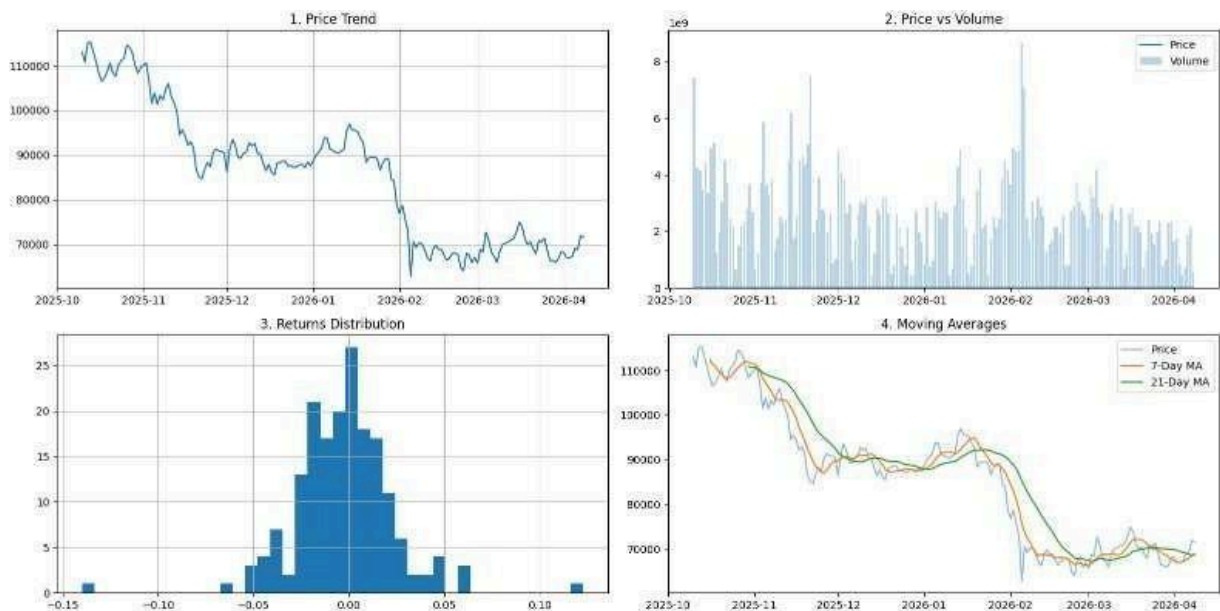


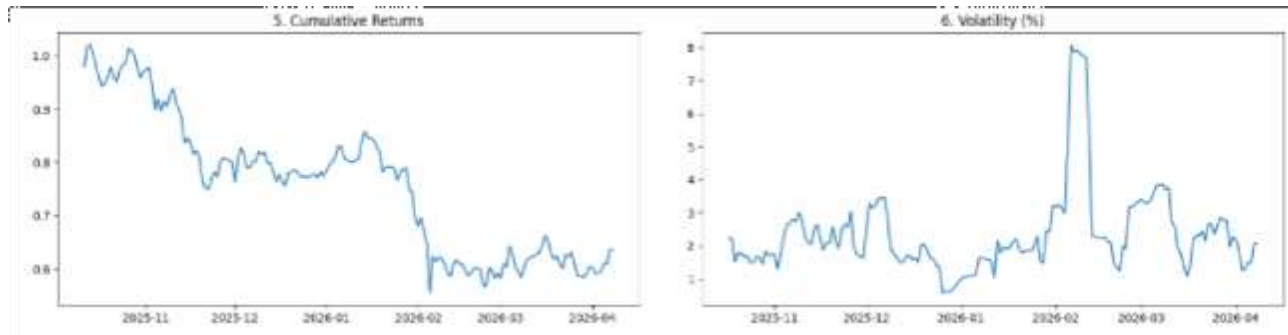
CoinMarketCap website

#	Name	Price	1h %	24h %	7d %	Market Cap	Volume(24h)	Circulating Supply
41	CoinMarketCap 20 Index DTF	\$147.34	+0.15%	+4.37%	+4.13%	\$15,202,093	\$7,247,000	103.17K CMC20
1	Bitcoin BTC	\$71,703.93	+0.32%	+3.88%	+4.85%	\$1,435,009,321,788	\$50,465,798,161	20.01M BTC
2	Ethereum ETH	\$2,251.70	+0.44%	+6.04%	+5.63%	\$271,760,366,637	\$26,468,410,099	120.69M ETH
3	Tether USDT	\$0.9998	+0.09%	+0.01%	+0.02%	\$184,127,897,752	\$99,772,434,008	184.15B USDT
4	XRP XRP	\$1.38	+0.05%	+4.73%	+3.31%	\$84,860,182,100	\$2,939,951,414	61.4B XRP
5	BNB BNB	\$612.04	+0.21%	+1.24%	+0.48%	\$83,456,440,261	\$2,210,011,624	136.35M BNB
6	USDC USDC	\$0.9998	+0.00%	+0.01%	+0.00%	\$77,898,265,470	\$12,428,745,336	77.92B USDC
7	Solana SOL	\$84.67	+0.19%	+5.55%	+1.63%	\$48,579,699,192	\$6,072,500,218	573.7M SOL
8	TRON TRX	\$0.3163	+0.02%	+0.42%	+0.44%	\$29,983,968,783	\$531,565,296	94.76B TRX
9	Dogecoin DOGE	\$0.09506	+0.22%	+4.20%	+2.95%	\$16,109,603,326	\$1,751,129,930	153.74B DOGE
10	Hyperliquid HYPE						\$346,165,100	256.09M HYPE



Dashboard :





Conclusion:-

By integrating automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, a comprehensive view of the cryptocurrency ecosystem can be achieved. This approach highlights the relationship between blockchain's underlying technical foundations and real-world market behavior, offering valuable insights into current trends as well as potential future developments in the digital asset industry.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

Exp 11

Title: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory:

Hyperledger Fabric is an open-source, permissioned blockchain platform developed for enterprise-level applications. Unlike public blockchains, where anyone can participate, Hyperledger Fabric restricts access to authorized users, providing enhanced security, privacy, and control over data. This makes it highly suitable for business environments where confidentiality and trust are critical.

Hyperledger Fabric follows a modular and flexible architecture. Its key components include **peers**, which maintain the distributed ledger and execute transactions; the **ordering service**, which ensures that all transactions are arranged in a consistent order; and **channels**, which allow a subset of network participants to create private communication layers for confidential transactions. This channel-based approach ensures that sensitive business data is only shared among relevant parties.

A core concept in Hyperledger Fabric is **chaincode**, which represents the business logic of the application. Chaincode is similar to smart contracts in other blockchain platforms and is typically written in programming languages like Go, Java, or JavaScript. It defines how assets are created, updated, transferred, and queried on the ledger. When a transaction is initiated, the chaincode is executed by endorsing peers, and the result is recorded on the ledger after validation.

The ledger in Hyperledger Fabric consists of two parts: the **blockchain**, which is an immutable record of all transactions, and the **world state**, which stores the latest state of assets for quick access. Assets can represent any tangible or intangible entity such as products, documents, or financial records, and they are managed efficiently through chaincode operations.

This framework ensures important features such as transparency, immutability, and security, while also providing high performance and scalability. As a result, Hyperledger Fabric is widely used in real-world applications like supply chain management, banking, healthcare, and digital identity systems, where secure and efficient asset management is essential.

Implementation:

Step 1: Update the System

```
sudo apt update  
sudo apt upgrade -y
```

```

g2022_mahvish_siddiqui@cloudshell:~$ sudo apt update
sudo apt upgrade -y
Get:1 https://cli.github.com/packages stable InRelease [3,917 B]
Get:2 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease [180 kB]
Get:3 https://apt.postgresql.org/pub/repos/apt noble-pgdg/main amd64 Packages [675 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:5 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,989 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,508 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [3,505 kB]
Get:9 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:10 https://packages.cloud.google.com/apt gcsfuse-noble InRelease [1,227 B]
Get:11 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [59.7 kB]
Get:12 https://packages.cloud.google.com/apt cloud-sdk InRelease [1,620 B]
Get:13 https://packages.cloud.google.com/apt gcsfuse-noble/main amd64 Packages [66.1 kB]
Get:14 https://packages.cloud.google.com/apt cloud-sdk/main all Packages [1,982 kB]
Get:15 https://packages.cloud.google.com/apt cloud-sdk/main amd64 Packages [4,640 kB]
Get:16 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease [3,005 B]
Hit:17 http://archive.ubuntu.com/ubuntu noble InRelease
Get:18 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:19 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0/multiverse amd64 Packages [68.7 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:21 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [3,692 kB]

```

Step 2: Install Required Dependencies

```
sudo apt install jq curl git nodejs npm -y
```

```
node -v
```

```
npm -v
```

```
git --version
```

```
docker
```

```
--version
```

```
docker ps
```

```

g2022_mahvish_siddiqui@cloudshell:~$ sudo apt install -y jq curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
curl is already the newest version (8.5.0-2ubuntu10.8).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
g2022_mahvish_siddiqui@cloudshell:~$ sudo apt install -y jq git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
g2022_mahvish_siddiqui@cloudshell:~$ █

```

```

g2022_mahvish_siddiqui@cloudshell:~$ node -v
v24.14.1
g2022_mahvish_siddiqui@cloudshell:~$ npm -v
11.12.1
g2022_mahvish_siddiqui@cloudshell:~$ git --version
git version 2.43.0
g2022_mahvish_siddiqui@cloudshell:~$ docker --v
unknown flag: --v

Usage:  docker [OPTIONS] COMMAND [ARG...]

Run 'docker --help' for more information
g2022_mahvish_siddiqui@cloudshell:~$ docker --version
Docker version 29.4.0, build 9d7ad9f
g2022_mahvish_siddiqui@cloudshell:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
g2022_mahvish_siddiqui@cloudshell:~$

```

Step 3: Clone Fabric Samples

git clone <https://github.com/hyperledger/fabric-samples>
cd fabric-samples

```

g2022_mahvish_siddiqui@cloudshell:~$ cd $HOME
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (159/159), done.
remote: Compressing objects: 100% (101/101), done.
remote: Total 15618 (delta 95), reused 58 (delta 58), pack-reused 15459 (from 4)
Receiving objects: 100% (15618/15618), 24.18 MiB | 16.03 MiB/s, done.
Resolving deltas: 100% (8760/8760), done.
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$

```

Step 4: Install Fabric Binaries and Images

curl -sSLO <https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh>

chmod +x install-fabric.sh

./install-fabric.sh docker samples binary

./install-fabric.sh docker

```

g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Full Hyperledger Fabric binaries

==> Downloading version 2.5.15 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
==> Will unpack to: /home/g2022_mahvish_siddiqui/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0     0     0     0     0     0     0     0  --:--:-- --:--:-- --:--:--    0
100 120M  100 120M    0     0  28.7M    0  0:00:04  0:00:04 --:--:--  31.0M
==> Done.

==> Downloading version 1.5.17 platform specific fabric-ca-client binary
==> Downloading: https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
==> Will unpack to: /home/g2022_mahvish_siddiqui/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0     0     0     0     0     0     0     0  --:--:-- --:--:-- --:--:--    0
100 31.6M  100 31.6M    0     0  22.3M    0  0:00:01  0:00:01 --:--:--  41.8M
==> Done.

```

```

Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
==> Pulling fabric ca Image
====> ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
==> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17         453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15      86dded7eda92      1GB       255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest             654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                 453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                 453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                 86dded7eda92      1GB       255MB
hyperledger/fabric-ccenv:2.5.15              86dded7eda92      1GB       255MB
hyperledger/fabric-ccenv:latest              86dded7eda92      1GB       255MB
hyperledger/fabric-orderer:2.5               9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15            9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest            9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                  3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15               3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest               3e25adc31a75      230MB      66.2MB
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$ 

```

Step 5: Verify Docker Images

docker images | grep fabric

```

g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17         453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15       3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                 654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15             654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                    453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                 453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                  86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15               86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB        255MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15             9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest             9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$

```

Step 6: Set Environment Variables

```

export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version

```

```

g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger

```

Step 7: Start Network and Create Channel

```

cd ~/fabric-samples/test-network
./network.sh up createChannel

```

```

g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$ cd ~/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb'
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/g2022_mahvish_siddiqui/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
ERROR: The Compose file './compose/compose-test-net.yaml' is invalid because:
Unsupported config option for services: 'peer0.org1.example.com'
Unsupported config option for volumes: 'peer0.org1.example.com'
Unsupported config option for networks: 'test'
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
Using docker and docker-compose

```

```

++ echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel", "type":2}}, "data":{"config_update":{"channel_id":"mychannel", "isolated_data": {}, "read_set": {}, "groups": {}, "Application": {"groups": {"Org2MSP": {"groups": {}, "mod_policy": "", "policies": {"Admins": {"mod_policy": "", "policy": null, "version": "0"}, "Readers": {"mod_policy": "", "policy": null, "version": "0"}, "Writers": {"mod_policy": "", "policy": null, "version": "0"}, "values": {"MSP": {"mod_policy": "", "value": null, "version": "0"}, "version": "0"}, "mod_policy": "", "policies": {}, "values": {}, "version": "0"}, "mod_policy": "", "policies": {"values": {}, "version": "0"}, "groups": {"groups": {}, "mod_policy": "", "policies": {"Admins": {"mod_policy": "", "policy": null, "version": "0"}, "Readers": {"mod_policy": "", "policy": null, "version": "0"}, "Writers": {"mod_policy": "", "policy": null, "version": "0"}, "values": {"AnchorPeers": {"mod_policy": "Admins", "value": {"anchor_peers": [{"host": "peer0.org2.example.com", "port": 9051}], "version": "0"}, "MSP": {"mod_policy": "", "value": null, "version": "0"}, "version": "0"}, "mod_policy": "", "policies": {"values": {}, "version": "0"}, "values": {"values": {}, "version": "0"}, "version": "0"}, "values": {}]]}}}}}}'
++ configtxlator proto encode --input /home/g2022_mahvish_siddiqui/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Envelope --output /home/g2022_mahvish_siddiqui/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-04-09 20:26:51.128 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-09 20:26:51.155 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$

```

Step 8: Deploy Chaincode

```

g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ ./network.sh deployCC \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccl javascript
Using docker and docker-compose
Deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged

```

Step 9: Set Peer Environment

```

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true

```

```
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"

export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 10: Initialize Ledger

peer chaincode invoke ... InitLedger

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com/ca.crt \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function":"InitLedger","Args":[]}'
2026-04-09 20:34:03.243 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 11: Query All Assets

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n
basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 12: Read Specific Asset

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n
basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 13: Create New Asset

peer chaincode invoke ... CreateAsset (Owner: Manav)

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 14: Transfer Asset

peer chaincode invoke ... TransferAsset

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt --peerAddresses localhost:9051 --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt -c '{"function": "InitLedger", "Args": []}'
2022-04-02 20:41:42.211 UTC [INFO] [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Step 15: Stop Network

./network.sh down

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ cd $HOME/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓ Container peer0.org1.example.com Removed 0.7s
✓ Container peer0.org2.example.com Removed 0.8s
✓ Container orderer.example.com Removed 0.6s
✓ Volume compose_peer0.org1.example.com Removed 0.0s
✓ Volume compose_peer0.org2.example.com Removed 0.0s
✓ Volume compose_orderer.example.com Removed 0.0s
✓ Network fabric_test Removed 0.1s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f6e64c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc1497615f5e33a8bfa908e0fd3555f864cfa1398b52d341c
b:latest
Deleted: sha256:12e3faf2b9ca30a7182645a005f0d09fe0fae610612b1f2bdfbe2799a1024009
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f6e64c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaeade1cf0b97f5c1163835f604aa885d60d
c:latest
Deleted: sha256:6fe9622fd07b0e414e4de896b19dae20f5b8312d5703581ca75bcc31a419608
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
481282afbc43: Pull complete
ae6ba0016ef2: Download complete
Digest: sha256:1497d0af5f52b4ba31c7e465126ee2123fe3f2305d638e7827681e7cf6c83d5e
```

Conclusion:

The experiment successfully demonstrated the implementation of a Hyperledger Fabric network along with deploying and interacting with chaincode for asset management. Through this process, we were able to create, query, and transfer assets efficiently, showcasing the core functionalities of a permissioned blockchain. It provided practical understanding of how transactions are executed, validated, and recorded securely within the network. Overall, the experiment highlighted the reliability, transparency, and control offered by Hyperledger Fabric for managing digital assets in a decentralized environment.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

BLOCKCHAIN CHARITY PLATFORM

Ethereum Smart Contract | Solidity | Decentralized Finance

1. What is Blockchain?

1.1 The Problem Blockchain Solves

Imagine you want to send money to someone in another country. Today, you need a bank. The bank sits in the middle of the transaction, charges fees, takes 3-5 days, and you have to trust them completely. What if the bank is dishonest? What if it goes bankrupt? What if it censors your transaction?

Blockchain solves this by removing the middleman entirely. Instead of one central authority (like a bank), thousands of computers around the world all share and verify the same record. This is called a decentralized system.

1.2 How a Blockchain Works

Think of blockchain like a public notebook that:

- Everyone in the world has a copy of
- Anyone can read from it, but writing to it requires everyone's agreement
- Once something is written, it can never be erased or changed
- Every new page (block) is mathematically linked to the previous page (chain)

If someone tries to cheat by altering a page, the math breaks, everyone notices, and the change is rejected. This makes blockchain tamper-proof and trustless - you don't need to trust any single person, because the math guarantees honesty.

1.3 What is Ethereum?

Bitcoin was the first blockchain - designed only for sending money. Ethereum (created in 2015) went further. Ethereum introduced Smart Contracts, which are programs that live on the blockchain and run automatically without any human intervention.

1.4 What is ETH and Wei?

Ethereum's currency is called Ether (ETH). Since ETH can be worth thousands of dollars, smart contracts work in tiny units called Wei (like paise vs rupees). The relationship is:

Unit	Value	Used For
1 ETH	1,000,000,000,000,000,000 Wei (10^{18})	Human-readable amounts
1 Gwei	1,000,000,000 Wei (10^9)	Gas fee calculations
1 Wei	Smallest possible unit	Internal contract math

In our project, goal is stored in Wei internally (goal = `_goalInEther` * 1 ether), but users interact in ETH for convenience.

2. Project Overview — What This Project Does

This project is a Decentralized Charity Crowdfunding Platform built on Ethereum. It allows anyone in the world to:

- Create a charity campaign with a funding goal and deadline
- Accept donations in ETH from anyone globally
- Automatically release funds to the beneficiary if the goal is reached
- Automatically refund all donors if the goal is NOT reached by the deadline

The most important thing about this platform is that it requires ZERO trust. The rules are coded in Solidity (Ethereum's programming language), deployed to the blockchain, and then run automatically forever. No human can interfere, steal the money, or change the rules after deployment.

2.1 Why This Matters — Real World Problem

Traditional charity platforms have serious problems:

- GoFundMe or similar platforms take 5-8% as fees
- You must trust the platform to forward your money honestly
- The platform can be hacked, go bankrupt, or become corrupt
- Refund processes are slow, manual, and unreliable
- There is no verifiable proof of where money went

Our blockchain solution eliminates ALL of these issues because:

- 0% platform fees (only Ethereum gas fees, which are tiny)
- No trust required - the smart contract guarantees behavior

- Cannot be hacked at the application level (math is unhackable)
- Refunds happen automatically and instantly through code
- Every transaction is public and permanently auditable on the blockchain

3. Code and Architecture

Code :-

```
pragma solidity ^0.8.0;
```

```
contract Charity {  
    string public name;  
    address payable public beneficiary;  
    uint public goal;  
    uint public deadline;  
    uint public totalDonations;  
    address public creator;
```

```
    mapping(address => uint) public donations;  
    mapping(address => bool) public refunded;  
    address[] public donors;
```

```
    bool private locked;  
    event DonationReceived(address indexed donor, uint amount);  
    event Withdrawn(address indexed beneficiary, uint amount);  
    event RefundIssued(address indexed donor, uint amount);
```

```
    modifier onlyBeneficiary() {  
        require(msg.sender == beneficiary, "Not beneficiary");  
        _;  
    }
```

```
    modifier onlyBeforeDeadline() {  
        require(block.timestamp <= deadline, "Deadline passed");
```

```
    _;  
}
```

```
modifier onlyAfterDeadline() {  
    require(block.timestamp > deadline, "Deadline not yet passed");  
    _;  
}
```

```
modifier nonReentrant() {  
    require(!locked, "Reentrant call");  
    locked = true;  
    _;  
    locked = false;  
}
```

```
constructor(  
    string memory _name,  
    address payable _beneficiary,  
    uint _goalInEther,  
    uint256 _durationInSeconds,  
    address _creator  
) {  
    name = _name;  
    beneficiary = _beneficiary;  
    goal = _goalInEther * 1 ether;  
    deadline = block.timestamp + _durationInSeconds;  
    creator = _creator;  
    totalDonations = 0;  
}
```

```
receive() external payable {  
    donate();  
}
```

```
function donate() public payable onlyBeforeDeadline {
```

```

    require(msg.value > 0, "Send ETH to donate");
    if (donations[msg.sender] == 0) {
        donors.push(msg.sender);
    }
    donations[msg.sender] += msg.value;
    totalDonations += msg.value;
    emit DonationReceived(msg.sender, msg.value);
}

function withdraw(uint amount) public onlyBeneficiary nonReentrant {
    require(totalDonations >= goal, "Goal not reached yet");
    require(amount <= address(this).balance, "Insufficient contract balance");
    (bool sent, ) = beneficiary.call{value: amount}("");
    require(sent, "Transfer failed");
    emit Withdrawn(beneficiary, amount);
}

function claimRefund() public nonReentrant onlyAfterDeadline {
    require(totalDonations < goal, "Goal reached - refunds disabled");
    uint amt = donations[msg.sender];
    require(amt > 0, "No donation to refund");
    require(!refunded[msg.sender], "Already refunded");
    refunded[msg.sender] = true;
    donations[msg.sender] = 0;
    (bool sent, ) = payable(msg.sender).call{value: amt}("");
    require(sent, "Refund transfer failed");
    emit RefundIssued(msg.sender, amt);
}

function getDonorCount() public view returns (uint) {
    return donors.length;
}

function getDonors() public view returns (address[] memory) {
    return donors;
}

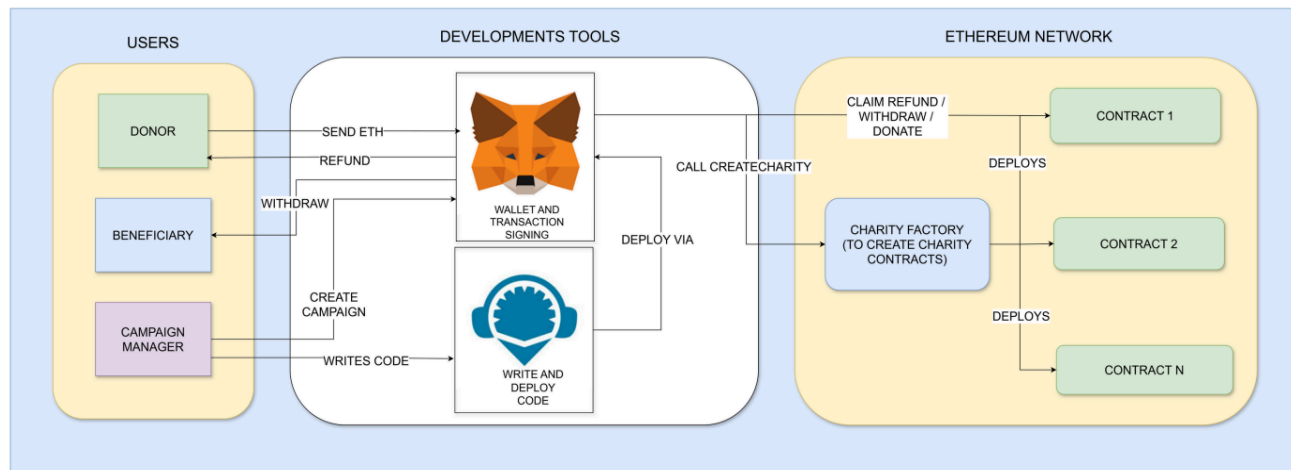
```

```
function getBalance() public view returns (uint) {  
    return address(this).balance;  
}
```

```
function getDonorAmount(address donor) public view returns (uint) {  
    return donations[donor];  
}  
}
```

```
contract CharityFactory {  
    address[] public charities;  
    event CharityCreated(address indexed charityAddress, string name, address beneficiary, uint  
goalWei, uint deadline);  
  
    function createCharity(  
        string memory name,  
        address payable beneficiary,  
        uint goalInEther,  
        uint durationSeconds  
    ) public returns (address) {  
        Charity newCharity = new Charity(name, beneficiary, goalInEther, durationSeconds, msg.sender);  
        charities.push(address(newCharity));  
        emit CharityCreated(  
            address(newCharity),  
            name,  
            beneficiary,  
            newCharity.goal(),  
            newCharity.deadline()  
        );  
        return address(newCharity);  
    }  
  
    function getCharities() public view returns (address[] memory) {  
        return charities;  
    }  
}
```

Contract	Role	Analogy
Charity	Manages ONE charity campaign	Like a single bank account for one fundraiser
CharityFactory	Creates & tracks multiple Charity contracts	Like a bank that opens new accounts for customers



This is called the Factory Design Pattern - one master contract (the Factory) that creates and keeps track of many child contracts (individual Charity campaigns). Each charity is its own independent smart contract with its own balance, rules, and state.

3.1 The Charity Contract — Deep Dive

3.1.1 State Variables — The Contract's Memory

State variables are data stored permanently on the blockchain. Every variable costs ETH to store because it takes up space on thousands of computers worldwide.

Variable	Type	What It Stores	Why It's Needed
name	string	Campaign name (e.g. 'Help Flood Victims')	Identifies the campaign to humans
beneficiary	address payable	Wallet address that receives funds	Knows who to pay when goal is reached
goal	uint	Target amount in Wei	Defines the fundraising target
deadline	uint	Unix timestamp of end time	Determines when campaign expires
totalDonations	uint	Sum of all donations received	Tracks progress towards goal

creator	address	Who created this campaign	Records the originator
donations	mapping	How much each address donated	Enables individual refunds
refunded	mapping	Whether each donor got refunded	Prevents double-refund attacks
donors	address[]	List of all donor addresses	Allows iterating over all donors
locked	bool	Reentrancy protection flag	Prevents certain types of attacks

```

contract Charity {
    string public name;
    address payable public beneficiary;
    uint public goal; // in wei
    uint public deadline; // unix timestamp
    uint public totalDonations;
    address public creator;

    mapping(address => uint) public donations;
    mapping(address => bool) public refunded;
    address[] public donors;

    bool private locked; // reentrancy guard
    event DonationReceived(address indexed donor, uint amount);
    event Withdrawn(address indexed beneficiary, uint amount);
    event RefundIssued(address indexed donor, uint amount);

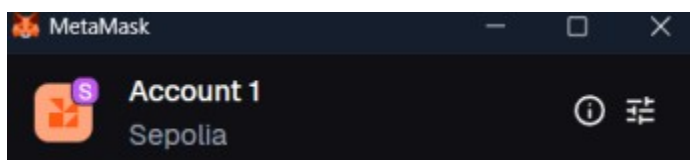
    modifier onlyBeneficiary() {
        require(msg.sender == beneficiary, "Not beneficiary");
        _;
    }
}

contract CharityFactory {
    address[] public charities;
    event CharityCreated(address indexed charityAddress, string name, address beneficiary);

    function createCharity(
        string memory name,
        address payable beneficiary,
        uint goalInEther,
        uint durationSeconds
    ) public returns (address) {
        Charity newCharity = new Charity(name, beneficiary, goalInEther, durationSeconds, msg.sender);
        charities.push(address(newCharity));
        emit CharityCreated(
            address(newCharity),
            name,
            beneficiary,
            newCharity.goal(),
            newCharity.deadline()
        );
        return address(newCharity);
    }

    function getCharities() public view returns (address[] memory) {
    }
}

```



3.1.2 Events — The Contract's Notifications

Events are notifications that the smart contract broadcasts to the outside world. They are stored in transaction logs on the blockchain. Frontend applications (websites, apps) can listen for these events and update the UI in real-time.

- `DonationReceived(donor_address, amount)` — Fires every time someone donates
- `Withdrawn(beneficiary_address, amount)` — Fires when the beneficiary withdraws funds
- `RefundIssued(donor_address, amount)` — Fires when a donor successfully claims a refund

Think of events as receipts. Even if the contract doesn't store the full history, events permanently log what happened and when.

3.1.3 Modifiers — Reusable Permission Guards

Modifiers are reusable rules that can be attached to any function. They run a check BEFORE the function executes. If the check fails, the entire transaction is rejected and reversed.

Modifier	What It Checks	If Check Fails...
onlyBeneficiary()	Is the caller the beneficiary address?	Reverts with 'Not beneficiary'
onlyBeforeDeadline()	Is current time $\leq$ deadline?	Reverts with 'Deadline passed'
onlyAfterDeadline()	Is current time $>$ deadline?	Reverts with 'Deadline not yet passed'
nonReentrant()	Is the locked flag false?	Reverts with 'Reentrant call'

4. Functions

4.1 constructor() — Birth of the Contract

The constructor runs ONCE when the contract is first deployed to the blockchain. It sets up all the initial values. After deployment, the constructor can never be called again.

Parameters it accepts:

- `_name` — The charity campaign name (e.g. 'School Building Fund')
- `_beneficiary` — The Ethereum address that will receive the funds if goal is met
- `_goalInEther` — Target in ETH (multiplied by 10^{18} internally to get Wei)
- `_durationInSeconds` — How long the campaign runs (e.g. 2592000 = 30 days)
- `_creator` — Address of who created this campaign (set by the Factory)

The deadline is cleverly computed as: `deadline = block.timestamp + _durationInSeconds`. `block.timestamp` is the current Unix time (seconds since Jan 1, 1970) provided by the blockchain itself.

4.2 donate() — Accepting Donations

This is the main function that accepts ETH from donors. It is marked payable, meaning it can receive ETH along with the function call.

Step-by-step execution flow:

1. Modifier check: `onlyBeforeDeadline()` — if campaign expired, reject immediately
2. `require(msg.value > 0)` — reject if no ETH was sent

3. Check if first-time donor — if `donations[msg.sender] == 0`, add to `donors[]` array
4. Add donation amount to their personal record: `donations[msg.sender] += msg.value`
5. Add to running total: `totalDonations += msg.value`
6. Emit `DonationReceived` event with donor address and amount

Note: The `receive()` function is a special fallback that automatically calls `donate()` if someone sends ETH directly to the contract address without specifying a function. This makes donating as simple as just sending ETH.

4.3 `withdraw()` — Beneficiary Collecting Funds

This function allows the beneficiary to withdraw ETH from the contract after the fundraising goal is reached.

Security checks (all must pass):

- `onlyBeneficiary()` — only the designated beneficiary can call this
- `nonReentrant()` — protects against reentrancy attack
- `totalDonations >= goal` — goal must be fully reached
- `amount <= address(this).balance` — can't withdraw more than contract holds

The transfer uses the low-level `.call{value: amount}("")` pattern which is the modern, recommended way to send ETH in Solidity. It returns a boolean (`sent`) that the code checks. If transfer fails for any reason, the entire transaction is reverted.

4.4 `claimRefund()` — Getting Money Back

If the campaign fails (deadline passes AND goal was not reached), donors can call this function to get their ETH back.

Security checks:

- `nonReentrant()` — prevents reentrancy attack (most critical here)
- `onlyAfterDeadline()` — can only refund after campaign ends
- `totalDonations < goal` — refunds only available if goal was NOT reached
- `donations[msg.sender] > 0` — caller must have actually donated
- `!refunded[msg.sender]` — caller cannot claim refund twice

The Checks-Effects-Interactions (CEI) Pattern is followed here — a critical security best practice:

7. CHECK: Verify all conditions are valid
8. EFFECT: Update state variables BEFORE sending money (`refunded[msg.sender] = true;`
`donations[msg.sender] = 0;`)
9. INTERACT: Only THEN send the ETH

By updating state before sending, even if the recipient is a malicious contract trying a reentrancy attack, the `refunded[msg.sender]` flag is already true, so the second call will fail.

4.5 View Functions — Reading Data

These functions only read data from the blockchain and cost no gas (ETH) to call. They are marked `views`.

Function	Returns	Purpose
<code>getDonorCount()</code>	<code>uint</code> (number)	How many unique addresses have donated
<code>getDonors()</code>	<code>address[]</code> (array)	Complete list of all donor wallet addresses
<code>getBalance()</code>	<code>uint</code> (Wei amount)	Current ETH balance held in the contract
<code>getDonorAmount(address)</code>	<code>uint</code> (Wei amount)	How much a specific address has donated

5. The CharityFactory Contract

5.1 How `createCharity()` Works

When someone calls `createCharity()` on the Factory:

10. A brand new Charity contract is instantiated with: `new Charity(name, beneficiary, goalInEther, durationSeconds, msg.sender)`
11. The new contract is deployed to the blockchain at its own unique address
12. The address is added to the `charities[]` array for record-keeping
13. A `CharityCreated` event is emitted with all campaign details
14. The address of the new Charity contract is returned to the caller

Note: `msg.sender` (the person calling `createCharity`) is passed as the `_creator` parameter to the Charity contract. This means the Factory correctly records WHO created each campaign.

5.2 `getCharities()` — Listing All Campaigns

This view function returns the complete array of all Charity contract addresses ever created through this Factory. A frontend application would call this to display all active campaigns to users.

6. Complete Lifecycle of a Campaign

Scenario A: Goal Reached — Success

Step	Who Acts	What Happens	State Change
1. Create	Anyone (creator)	Calls CharityFactory.createCharity()	New Charity contract deployed
2. Donate	Donors (anyone)	Send ETH to donate()	totalDonations increases
3. Goal Met	Automatic	totalDonations reaches goal	withdraw() becomes available
4. Withdraw	Beneficiary	Calls withdraw(amount)	ETH sent to beneficiary
5. Done	—	Campaign is complete	Balance reduces to 0

Scenario B: Goal Not Reached — Refunds

Step	Who Acts	What Happens	State Change
1. Create	Anyone (creator)	Calls CharityFactory.createCharity()	New Charity contract deployed
2. Donate	Donors	Send ETH to donate()	totalDonations increases (but below goal)
3. Deadline Passes	Automatic (time)	block.timestamp > deadline	No more donations possible
4. Refund	Each donor	Calls claimRefund()	Their ETH returned, record zeroed out
5. Done	—	All donors refunded	Contract balance reaches 0

7. Security Features

Security Feature	Where Used	What Attack It Prevents
------------------	------------	-------------------------

Reentrancy Guard (nonReentrant)	withdraw() and claimRefund()	Reentrancy attack — malicious contracts calling back in
Checks-Effects-Interactions Pattern	claimRefund()	Reentrancy attack — state updated before ETH transfer
Double-refund Protection (refunded map)	claimRefund()	Same donor claiming refund twice
Modifier-based Access Control	withdraw() — onlyBeneficiary	Unauthorized fund withdrawal
Deadline Enforcement	donate() — onlyBeforeDeadline	Donations after campaign ends
Zero-value Rejection	donate() — require(msg.value > 0)	Spam / zero-value calls
Balance Check	withdraw() — amount <= balance	Overdraft / insufficient funds

8. How to Run & Deploy This Project

8.1 Tools You Need

- Remix IDE — Browser-based Solidity IDE. No installation needed. Open remix.ethereum.org
- MetaMask — Browser extension wallet. Download from metamask.io
- Sepolia Testnet — A test version of Ethereum where ETH is free (no real money)
- Sepolia Faucet — A website that gives you free test ETH (cloud.google.com/application/web3/faucet/ethereum/sepolia)

8.2 Step-by-Step Deployment Guide

Step 1: Set Up MetaMask

15. Install MetaMask browser extension and create an account
16. Switch network from 'Ethereum Mainnet' to 'Sepolia Testnet'
17. Visit the Sepolia faucet, paste your wallet address, receive free test ETH

Step 2: Deploy in Remix IDE

18. Open remix.ethereum.org in your browser
19. Create a new file called Charity.sol and paste the entire code

20. Click 'Solidity Compiler' tab → Select version 0.8.0 or higher → Click 'Compile'
21. Click 'Deploy & Run Transactions' tab
22. Change Environment from 'Remix VM' to 'Injected Provider — MetaMask'
23. Select 'CharityFactory' from the contract dropdown
24. Click 'Deploy' → MetaMask will pop up → Confirm the transaction
25. Wait ~15 seconds → CharityFactory is now live on Sepolia blockchain!

Step 3: Create a Charity Campaign

26. In Remix, expand the deployed CharityFactory contract
27. Click 'createCharity' and fill in the parameters:
 - name: "Feed Stray Dogs"
 - beneficiary: 0xYourEthereumAddressHere
 - goalInEther: 1
 - durationSeconds: 604800 (= 7 days)
28. Confirm in MetaMask → A new Charity contract is deployed at a unique address

Step 4: Donate to the Campaign

29. Call getCharities() on the Factory to see the Charity contract's address
30. In Remix, load that Charity contract using 'At Address' button
31. In the top VALUE field, type 0.5 (ETH) and select 'ether' from dropdown
32. Click 'donate' → MetaMask confirms → You've donated 0.5 ETH!

Step 5: Verify the Donation

- Call totalDonations — should show 500000000000000000 (0.5 ETH in Wei)
- Call getBalance — should match totalDonations
- Call getDonorCount — should return 1
- Call getDonorAmount with your address — should show your donation

Step 6: Withdraw (if Goal Reached)

- Call withdraw with amount = 500000000000000000 (0.5 ETH in Wei) from the beneficiary account
- MetaMask will pop up for confirmation
- Once confirmed, the ETH moves to the beneficiary wallet

9. Testing Scenarios

Test Case	How to Test	Expected Result
Happy path donation	Send ETH via donate() before deadline	totalDonations increases, event emitted
Donation after deadline	Advance time past deadline, try to donate	Transaction reverts: 'Deadline passed'
Withdraw before goal	Try withdraw() when totalDonations < goal	Transaction reverts: 'Goal not reached yet'
Unauthorized withdraw	Call withdraw() from non-beneficiary account	Transaction reverts: 'Not beneficiary'
Successful refund	Let campaign expire with goal unmet, call claimRefund()	ETH returned to donor
Double refund attempt	Call claimRefund() twice from same address	Second call reverts: 'Already refunded'
Refund with goal met	Reach goal, then try claimRefund()	Transaction reverts: 'Goal reached - refunds disabled'
Zero value donation	Call donate() with 0 ETH	Transaction reverts: 'Send ETH to donate'

10. Conclusion

A charity system using the Blockchain concept was successfully built and programmed in the Solidity language. Ether (ETH) cryptocurrency was used to donate and for the charity to function.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 13

Aim: Assignment 1

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 14

Aim: Assignment 2

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Questions ● Draw a neat diagram and explain components of Hyperledger Fabric? ● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric? ● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus ● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token ● What is Tokenization? Explain its significance and challenges/limitation with example ● Compare Fungible and non Fungible tokens (Give examples of each) ● How can blockchain-based IoT networks improve device management, data monetization, and user privacy in the rapidly growing IoT ecosystem? ● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.
CO Mapped	CO4,CO5,CO6

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 15

Aim: Assignment 3

Roll No.	59
Name	Jai Talreja
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Q1 Analyze & compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake & PBFT in terms of security, scalability, energy efficiency & fault tolerance. Illustrate your answer with examples like Bitcoin & Ethereum. Q2 Analyze different types of nodes in a blockchain network & explain their role in maintaining the system. Compare how each node contributes to validation, storage & network efficiency. (Compare UTXO model) Q3 Analyze the UTXO model & explain how it ensures transaction transparency & security in blockchain systems. Illustrate with an example such as Bitcoin. Q4 Explain crypto asset & cryptocurrency. Examine & analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance & decentralized applications. Q5 Analyze the role of payment scripts in blockchain transactions. Examine & analyze different types of payment scripts. Explain how they ensure secure & verifiable transfer of coins. Illustrate with an example. Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks & opportunities of smart contracts. Q7 Explain the structure of BTC transaction & ETH transaction. Q8 Explain the difference between mining & minting and explain the process
CO Mapped	CO1- CO6

Hard Copy Submitted